

Radical Denesting after the Second Repair

A mathematical and implementation audit
of `StradFixed2.wl`

Type boundaries, monotone search, branch certificates,
and field-aware candidate construction

Prepared for Vladimir Reshetnikov
September 5, 2026

Assessment. The second corrected implementation has a substantially better acceptance architecture than its predecessors. Its remaining problems are chiefly an overpermissive algebraicity predicate, nonmonotone search-stage handoffs, incomplete index-reduction admission, and resource-dependent negative caching. This audit does not claim a reproduced wrong default numerical output.

Delivered proposal. A self-contained `StradFixed3.wl` derivative retains the principal algorithms, repairs those contracts, and adds bounded higher-odd-index and square-class-coset searches. It uses a separate package context for side-by-side testing.

Validation boundary. The accompanying Python run passed 3,399 exact mathematical assertions, three control-flow-model assertions, and ten lexical/structural assertions. The 52 supplied native Wolfram tests were *not executed*: the connector failed with HTTP 404 and no local kernel was available. The proposed package requires native regression validation before adoption.

Source snapshot. The current source and prior unified-B analysis were inspected through GitHub/raw web views on September 5, 2026. The snapshot was not resolved to an immutable commit. Primary literature was consulted through accessible author-hosted copies; the complete downloaded literature corpus was not independently audited.

Abstract

Radical denesting separates naturally into candidate construction, branch-sensitive equality certification, and a representation objective. A program may be conservative at its final acceptance gate yet still discard useful candidates, misclassify its input domain, or obscure the distinction between an unsuccessful bounded search and a mathematical negative result. This article audits those boundaries in `StradFixed2.wl`, explicitly distinguishing new source-level findings from limitations already acknowledged by the unified-B review.

The main contract defect is the treatment of an exact Root head as sufficient evidence of algebraicity, although Wolfram Language permits roots with nonalgebraic coefficients. The search analysis identifies incumbent resets, a requirement that recursive index reduction change an already useful intermediate expression, and memoization that makes a low-resource failure suppress later exploration. Separate sections analyze numerical rejection, cooperative budgets, traversal, cost noncompositionality, and report semantics.

The mathematical development verifies the direct and indirect quadratic formulas, gives a complete trace–norm reconstruction criterion for odd roots in a real quadratic field, derives the Honsbeek numerator identity with its branch obligations, and proves a square-class-coset lemma. An explicit three-surd example shows why even closing the input square classes is insufficient: the desired square root can live in a different coset. The supplied implementation uses these results as candidate generators behind an exact acceptance gate. Mathematical verification and static checks are reported separately from the native execution that remains outstanding.

Contents

1	Evidence, scope, and interpretation	2
2	Architecture and the contract worth preserving	3
3	Findings at a glance	4
4	Type safety and equality classification	5
5	Search monotonicity, index reduction, and memoization	6
6	Mathematical audit of the quadratic fast paths	8
7	Trace–norm reconstruction for arbitrary odd index	9
8	Honsbeek’s quartic: identity, recognition, and branches	10
9	Square-class cosets: a concrete gap and a bounded repair	11
10	Multiplier search, polynomial GCDs, and rationalization	13
11	Budgets, traversal, costs, and observability	15
12	The proposed <code>StradFixed3.wl</code>	17
13	Verification performed and the native release gate	18
14	Prioritized development directions	20
15	Conclusion	20
A	Reproduction commands and archive map	21
B	Selected regression obligations	21
C	Complete proposed implementation	23

1 Evidence, scope, and interpretation

1.1 What was inspected

The principal object is the current `src/corrected/StradFixed2.wl` source in the RadicalDenest repository [1]. The comparison baseline is the current `src/code-review/unified-B/unified_analysis_B.tex` [2], rather than an earlier review of `StradFixed.wl`. The repository’s unified literature guide supplies the broader organization of denesting questions [3]. The relevant primary mathematical sources consulted include Borodin–Fagin–Hopcroft–Tomba, Gkioulekas, and Honsbeek [4, 5, 6]; Jeffrey–Rich is an additional algorithmic reference [7]. Official Wolfram documentation is used for language contracts, not as evidence that this particular package was executed.

Initial raw downloads and direct repository access were unreliable. Eventually the source and prior review were recovered in web-rendered/raw views. The current report was read in substantive sections. The literature directory itself was visible, but its full manifest and child corpus could not be retrieved reliably; accessible primary copies of the relevant works were used instead. No claim is made to have checked every bibliography entry, every paper in the literature directory, or every execution log in unified-B.

This archive is consequently a source-informed derivative, not a byte-for-byte patch against a locally cloned, commit-pinned checkout. Function names and short code anchors identify findings. Apparent line numbers in web renderings are not substituted for physical file line numbers. The source-access ledger records the URLs and limits, and the archive hashes identify the delivered proposal, not the upstream file.

1.2 Four different kinds of evidence

Source-level findings follow from inspected definitions and control flow. **Mathematical findings** are established by the proofs and exact identities below. **Independent computational checks** are the actually executed SymPy and structural checks in the archive. **Native regression expectations** are Wolfram tests supplied for execution, not observed outputs.

These distinctions matter. A helper can violate an incumbent-preservation contract even when another stage later recovers the same improvement. A type predicate can admit a transcendental object without thereby demonstrating that the final denester returns a wrong value. Conversely, thousands of exact identity checks in Python do not establish correct Wolfram pattern matching, scoping, option evaluation, message handling, or package loading.

1.3 What the prior report already establishes or acknowledges

The unified-B analysis reports native execution on Wolfram Language 15.0.1 for Windows, a 116-case battery with 71 depth reductions and no wrong results or messages, a further 340 structured randomized cases without wrong results, and 144 passing regression tests [2]. These are *repository-reported results*, not reruns performed for this article. They are useful baseline evidence, but the proposed derivative cannot inherit their pass status automatically.

The prior report also expressly discusses heuristic numerical rejection, per-island trial budgets, conservative treatment of mixed exact/inexact input, noncompositional costs, restricted multi-surd bases, a restricted Honsbeek recognizer, cooperative resource limits, and the fact that certificate records are not standalone proof objects. Those are not presented here as previously unnoticed omissions. Where an optional alternative is proposed, it is identified as a policy change or a coverage extension.

2 Architecture and the contract worth preserving

2.1 The three independent questions

Let E be an expression whose evaluated value is a finite algebraic number. A candidate C must answer three questions:

$$\begin{aligned} & \text{Is } C \text{ in the output grammar?} \\ & \text{Does } C = E \text{ as a selected complex value?} \\ & \text{Is } C \text{ preferable under the stated cost?} \end{aligned} \tag{2.1}$$

None implies the others. A Root representation may certify an algebraic value without displaying a denested radical [10]. Equal minimal polynomials do not identify the same conjugate. A mathematically correct rewrite may increase the chosen representation cost. The literature guide properly separates existence, construction, minimum depth, equality testing, and practical computation [3]; the code contract should retain that separation.

2.2 The inspected pipeline

The public entry points resolve options, initialize a dynamically scoped session, traverse permitted arithmetic hosts, and process exact numerical islands. Within an island the main sequence is shape-specific formulas, negative-radical handling, linear-factor/index-reduction searches, generic algebraic simplification, and a bounded multiplier queue. Candidates pass through local acceptance and polishing routines. Repeated passes are bounded, and a whole-expression cost check can reject a locally improved reconstruction [1].

The representation cost is a lexicographically ordered tuple of opaque algebraic-object count, maximum radical depth, radical-node count, LeafCount, and rational bit size. This is a reasonable explicitly chosen heuristic, not an invariant of algebraic numbers. Counting Root objects first expresses a preference for visible radical formulas; it does not establish that the resulting formula has minimum depth or minimum arithmetic complexity.

2.3 Repairs that should not be undone

The second version's local equality gate is essential. In particular, a custom solver's proposal for a numerical island under a symbolic host is not accepted merely because it looks simpler. Ambient assumptions are isolated, and speculative branch manipulations are checked against the selected exact value. Failure sentinels are not used as mathematical polynomials. The polynomial-GCD path retains candidates through degree four rather than discarding the cubic and quartic cases. The multiplier admission cap is centralized [1, 2].

The cost comparator using Order is not reversed: Wolfram documents $\text{Order}[a, b] == 1$ when a precedes b in canonical order, which is the intended comparison for these fixed-length numeric cost tuples [12]. Similarly, the indirect quadratic condition discussed below involves $-cD$, not cD . A review should not reintroduce an old algebraic sign error or discard a working quartic path in pursuit of superficial simplification.

2.4 Conditional soundness invariant

Proposition 2.1 (Local acceptance and arithmetic reconstruction). *Suppose every replaced numerical subtree is replaced by an exactly equal selected value, and reconstruction uses ordinary pure arithmetic operations*

or lists. Then reconstruction preserves the expression’s value wherever those operations are defined. If, in addition, every committed pass strictly decreases a well-founded representation objective, no infinite sequence of committed improvements is possible.

Proof. The first statement is substitution of equal values into the same deterministic operation, proved inductively up the expression tree. Principal powers do not invalidate this argument: the same base value and exponent determine the same principal value. What would be invalid is replacing a product of powers by a power of a product without checking that branch-sensitive identity. For the second statement, the nonnegative integer cost tuples are well-founded under lexicographic order with fixed length. An infinite strictly decreasing sequence cannot exist. $\square$

This is a mathematical argument about pure arithmetic trees, not a security guarantee about arbitrary user definitions, callbacks, side effects, or maliciously constructed pure functions inside opaque heads. Explicit pass, recursion, trial, and time limits remain necessary because a stage can spend a long time without committing any improvement.

3 Findings at a glance

ID	Classification	Finding	Disposition in proposal
F01	High: domain contract	Opaque heads and infinite precision do not establish algebraicity.	Validate payloads or checked canonicalization.
F02	Medium: search logic	Stage assignments can discard an existing better candidate.	Thread the incumbent through every stage.
F03	Medium: helper coverage	Index reduction requires recursion to change a useful intermediate result.	Offer raw and recursive candidates.
F04	Medium: search state	Expression-only memoization retains resource-dependent failures.	Positive-only incumbent cache.
F05	Medium: API semantics	Heuristic rejection is exposed as an exact “Different” status.	Separate exact status from numerical rejection.
F06	Medium: resource contract	Some expensive operations and final costs escape their advertised local budgets.	Wider session scope; explicit operation wrappers.
F07	Known limit; extension	Two multi-surd bases miss entire square-class cosets.	Optional bounded coset search.
F08	Known limit; extension	Honsbeek recognition is narrower than its sufficient identity.	Recognize rational cubes of terms; certify branches.
F09	Algorithmic extension	The quadratic trace–norm fast path stops at cubes.	Bounded general odd-index reconstruction.
F10	Known policy	Local depth improvement need not improve a composite cost.	Preserve global default; optional independent lists.
F11	Known policy	One inexact number disables exact-island processing throughout a host.	Preserve default; optional exact-island mode.
F12	Efficiency; minor cap	Syntactic multiplier keys miss equivalences; an inner batch can exceed its nominal quota.	Optional canonical keys; inner-loop checks.
F13	Observability	Accepted-candidate records are not a final-result proof chain.	Separate journals and truncation flags.
F14	Minor interface	Some malformed argument forms never reach validation.	Explicit public argument dispatch.

The severity labels refer to contract importance and search quality. None is a claim of an observed wrong default numerical result in a native run of the current source.

4 Type safety and equality classification

4.1 F01: An exact Root object need not be algebraic

The original predicate recognizes `Root` and `AlgebraicNumber` as opaque algebraic heads and tests exact precision and absence of machine-real payloads. That is insufficient for `Root`. Its official documentation explicitly allows exact numeric nonalgebraic coefficients and symbolic parameters [8]. Consider the intended object

```
r = Root[#^3 + Pi # + 1 &, 1];
RadicalDenest2'ExactAlgebraicQ[r]
```

The supplied native probe checks the actual representation and predicate result on the installed kernel; it was not executed here. The mathematical obstruction is unconditional: every root z of

$$z^3 + \pi z + 1 = 0 \tag{4.1}$$

is transcendental. Indeed, $z \neq 0$, and algebraicity of z would imply $\pi = -(z^3 + 1)/z$ is algebraic. Infinite precision is a representation property, not a proof that coefficients belong to $\overline{\mathbb{Q}}$.

Likewise a root containing a free parameter is not an explicit exact algebraic *constant*. Merely recognizing the outer head fails to enforce the advertised input domain. This matters because the exact algebraic zero-testing guarantee is conditional on genuinely explicit algebraic input [9].

Repair. The proposal checks a conventional univariate `Root`'s rational defining polynomial, positive integral root index, finite exact payload, and numeric character. A rational-coefficient polynomial is enough to prove algebraicity; irreducibility is not required. Algebraic-coefficient or triangular root representations may instead be admitted after bounded `RootReduce` conversion into a checked rational grammar. An `AlgebraicNumber` requires an accepted base and rational coefficient list. Unsupported representations return a conservative negative from the recognizer, and the equality API returns "Unknown" rather than relying on an unproved type assertion.

There is a tradeoff: this recognizer is deliberately not a complete decision procedure for every exact expression the kernel can represent. It also evaluates defining functions inside a resource wrapper, so it is not a sandbox for hostile code hidden in those functions. A production API could use a richer result such as "SupportedAlgebraic", "Unsupported", and "ResourceLimit", instead of overloading a Boolean.

4.2 F05: Numerical rejection should not masquerade as exact inequality

In the inspected classifier, a high-precision numerical discrepancy can produce "Different" before the exact algebraic reduction is attempted. The unified-B report knowingly retains this filter for speed and correctly observes that a false rejection cannot introduce a wrong accepted rewrite [2]. That defense is sound as far as acceptance is concerned. It does not make the public status an exact proof of inequality.

Accuracy estimates and a fixed discrepancy threshold are not interval enclosures. The distinction is especially important when the usage text describes a result as decided by exact algebra. The proposal therefore places optional numerical rejection in the candidate-search layer and counts it separately. Public `EqualityStatus` uses checked algebraic input, exact reduction, and the documented

PossibleZeroQ[... , Method -> "ExactAlgebraics"] path; failure to establish a Boolean result yields "Unknown" [9].

A subtle regression must be avoided here. RootReduce need not turn every nonzero algebraic expression into an atomic Root. A result such as $2\sqrt{2}$ is nonzero but remains composite. The prior review reports exactly this weakness in one earlier proposal. The new classifier consequently retains the exact zero-test fallback, rather than recognizing nonzero values only when the reduction is an integer, rational, complex atom, or Root.

4.3 Branch equality is stronger than a shared polynomial

For example, $\sqrt{2} + \sqrt{3}$ and $\sqrt{3} - \sqrt{2}$ satisfy the same polynomial $x^4 - 10x^2 + 1$ but are unequal. Similarly, equality of q th powers determines a value only up to a q th root of unity. A candidate-generation identity is not itself a selected-root certificate. The gate must test the actual principal-valued expressions, or a future independent checker must include sufficient isolating and branch information. Power uses principal complex values, so a negative real cube root in elementary real notation is not automatically the same expression as a principal fractional power [11].

5 Search monotonicity, index reduction, and memoization

5.1 F02: Incumbent-reset assignments violate the search contract

The intended meaning of an incumbent is the best certified result found so far. However, the inspected powerProposals assigns the results of the negative-radical and Kummer helpers directly to best, while those helpers initialize from, or fall back to, the original target. Schematically:

```
best = incumbent;
(* earlier candidates are tried *)
best = negativeRadicalCandidate[target, rho, p, q];
best = kummerCandidates[target, rho, p, q];
```

A failed or inapplicable stage is therefore not the identity on the incoming incumbent. The negative-radical stage can discover a partial improvement that is subsequently discarded when the next stage returns the target [1].

The scope of this finding should be stated precisely. In the current fast paths, a successful direct formula commonly produces depth one and triggers an early return, so the first reset does not ordinarily erase those particular successes. A contract test with a supplied partial incumbent establishes the reset problem; it is not evidence that a familiar square-root example returns a wrong result. Later generic simplification or multiplier search can also rediscover a lost candidate. The defect is loss of guaranteed search progress and avoidable order dependence.

Repair. Give every stage an incumbent argument and initialize its local best value from that argument. Every candidate is compared with the incumbent by the same acceptance gate. Then a stage that finds nothing returns the incumbent unchanged. This convention is simpler to reason about than attempting to repair the caller after each assignment.

5.2 F03: An already useful index reduction is withheld from acceptance

The Kummer helper factors $x^k - \rho$, with $k \mid q$, and uses a linear-factor root γ to replace a q th-root problem by a (q/k) th-root problem. In the residual-index branch, the current source only offers a candidate when recursion changes the residual expression *and* strictly lowers radical depth. This excludes an intermediate candidate that is already cheaper, or a same-depth improvement under later components of the advertised cost.

A small mathematical example is

$$T = (3 + 2\sqrt{2})^{1/6}, \quad B = (1 + \sqrt{2})^{1/3}. \quad (5.1)$$

Since $3 + 2\sqrt{2} = (1 + \sqrt{2})^2$ and both bases are positive, $T = B$. Both displayed expressions have depth two, but the latter removes the inner coefficient and shortens the representation. The reduction with $k = 2$ supplies B before recursion. With a zero recursion allowance, or when recursion leaves B unchanged, the original helper does not offer that candidate through the residual branch. This is a helper-level coverage statement, not a claim that the full public function cannot find the identity by another route.

Repair. Offer the raw residual candidate, with all necessary phase choices, before attempting recursion. Offer the recursive result afterwards as well. Let the full cost tuple, not a preliminary depth-only test, decide progress. The analogous even-index split should use the same rule.

5.3 F04: A bounded-search failure is not an expression property

The source memoizes the best result for a target even when that best result is the unchanged target. Yet recursive calls reduce the available trial quota, and admissible recursion depth depends on the call context. A failure encountered under those conditions can be reused later when more search would have been possible. Expression-only negative caching thus confuses “not found in this search” with “no improvement available” [1].

A correct negative cache would need to record what search region was exhausted, with which options and budgets, and whether a resource interruption occurred. Most heuristic denesters cannot attach a useful mathematical nonexistence certificate to such a record. The proposal takes the simpler approach: cache only certified positive progress, and use it as an incumbent. Do not cache an unchanged result or an “Unknown” equality result as a mathematical negative. An active-call guard prevents immediate recursive cycles, and trial usage is tracked per target during the session.

This change can increase repeated work. That is an explicit performance tradeoff, not a claim of universal acceleration. A subsequent optimization should cache exact reusable subcomputations—factorizations, canonical algebraic representations, or successful equality certificates—rather than resource-dependent unsuccessful searches.

5.4 Why these changes help independently of final equality

The exact gate protects value correctness but cannot recover candidates that never reach it or were overwritten. Monotone stage interfaces and budget-aware state are therefore not redundant with certification. They establish a different invariant: once a valid improvement is found, adding a later unsuccessful stage does not make the final incumbent worse.

6 Mathematical audit of the quadratic fast paths

Let

$$\rho = a + b\sqrt{c} > 0, \quad a, b, c \in \mathbb{Q}, \quad c > 0, \quad \sqrt{c} \notin \mathbb{Q}, \quad b \neq 0, \quad D = a^2 - b^2c. \quad (6.1)$$

The representation recognizer may absorb a rational coefficient into c ; that does not alter the following identities. Direct and indirect denesting are different restricted problems, as emphasized in the classical square-root literature [4, 5, 7].

6.1 Direct square-root denesting

Suppose $D = s^2$ for a rational $s \geq 0$ and $a > 0$. Define

$$u = \frac{a + s}{2}, \quad v = \frac{a - s}{2}. \quad (6.2)$$

Then $u, v \geq 0$ and $uv = b^2c/4$. Consequently

$$\sqrt{a + b\sqrt{c}} = \sqrt{u} + \operatorname{sgn}(b)\sqrt{v}. \quad (6.3)$$

To verify the selected sign, the square of the right side is $a + b\sqrt{c}$. For $b > 0$ it is positive. For $b < 0$, $u \geq v$ and the difference is nonnegative; strict positivity of ρ excludes a zero value. Thus the principal square root is obtained, not merely one of the two square roots.

Conversely, a representation $\sqrt{u} + \varepsilon\sqrt{v}$ with rational nonnegative u, v and the indicated quadratic irrational cross term gives $D = (u - v)^2$. This converse concerns that restricted two-square-root form; it is not a universal impossibility theorem for arbitrary radical languages.

6.2 Indirect fourth-root denesting

Assume $D < 0$, $b > 0$, and

$$e = \sqrt{b^2 - a^2/c} \in \mathbb{Q}. \quad (6.4)$$

Since $0 \leq e \leq b$, the radicands below are nonnegative. Then

$$\sqrt{a + b\sqrt{c}} = c^{1/4} \left(\sqrt{\frac{b + e}{2}} + \operatorname{sgn}(a)\sqrt{\frac{b - e}{2}} \right). \quad (6.5)$$

Squaring the parenthesis gives $b + a/\sqrt{c}$, because $\sqrt{b^2 - e^2} = |a|/\sqrt{c}$. Multiplication by $\sqrt{c}$ yields ρ . The chosen difference, when $a < 0$, is nonnegative because the first radicand is at least the second. The $a = 0$ case reduces correctly because $e = b$.

The rationality condition is equivalently that $-cD$ is a rational square:

$$-cD = c^2 (b^2 - a^2/c). \quad (6.6)$$

The minus sign is essential. This verifies the sign convention retained by the corrected source. The role of fourth roots in this local classification should not be extrapolated to every deeper nesting problem [4, 5].

6.3 Gaussian square roots

For $z = a + ib$, $a, b \in \mathbb{Q}$, with $b \neq 0$, suppose $n = \sqrt{a^2 + b^2} \in \mathbb{Q}$. Then

$$\sqrt{z} = \sqrt{\frac{n+a}{2}} + i \operatorname{sgn}(b) \sqrt{\frac{n-a}{2}}. \quad (6.7)$$

The real part is nonnegative, the imaginary part has the sign of b , and squaring gives $a + ib$. Purely real inputs are handled separately; a negative real number must not be fed into a formula that silently sets the imaginary contribution to zero because $\operatorname{sgn}(0) = 0$. In the source, the Gaussian helper is selected by an actual complex-number head, while real negative radicands follow the phase path.

7 Trace–norm reconstruction for arbitrary odd index

The original cubic fast path is mathematically sound when both its norm and rational-trace conditions are enforced. Its natural extension is not an enumeration of integer divisors of a norm, but a polynomial identity in trace and norm. The following derivation supplies the proposed "HigherOddQuadratic" path.

7.1 Two polynomial sequences

Define polynomials in T, n by

$$D_0 = 2, \quad D_1 = T, \quad D_m = TD_{m-1} - nD_{m-2}, \quad (7.1)$$

$$S_0 = 0, \quad S_1 = 1, \quad S_m = TS_{m-1} - nS_{m-2}. \quad (7.2)$$

If $u + v = T$ and $uv = n$, induction gives

$$D_m = u^m + v^m, \quad S_m = \frac{u^m - v^m}{u - v} \quad (u \neq v). \quad (7.3)$$

It follows that

$$D_m(T, n)^2 - 4n^m = (T^2 - 4n)S_m(T, n)^2. \quad (7.4)$$

This is a polynomial identity and therefore also holds on the repeated-root locus $T^2 = 4n$. For example,

$$D_3 = T^3 - 3nT, \quad S_3 = T^2 - n, \quad (7.5)$$

$$D_5 = T^5 - 5nT^3 + 5n^2T, \quad S_5 = T^4 - 3nT^2 + n^2. \quad (7.6)$$

Theorem 7.1 (Odd roots in a real quadratic field). *Let $q \geq 3$ be odd, let $c > 0$ be a rational nonsquare, and let $a, b \in \mathbb{Q}$ with $b \neq 0$. There is a real $\beta \in \mathbb{Q}(\sqrt{c})$ satisfying $\beta^q = a + b\sqrt{c}$ if and only if there exist $n, T \in \mathbb{Q}$ such that*

$$n^q = a^2 - b^2c, \quad D_q(T, n) = 2a. \quad (7.7)$$

For every such pair, $S_q(T, n) \neq 0$, and the required value is

$$\beta = \frac{T}{2} + \frac{b}{S_q(T, n)}\sqrt{c}. \quad (7.8)$$

Proof. If $\beta = A + B\sqrt{c}$, its conjugate is $\beta' = A - B\sqrt{c}$. Put $T = \beta + \beta' = 2A$ and $n = \beta\beta' = A^2 - cB^2$. Taking norm and trace of β^q proves (7.7).

Conversely, substitute (7.7) into (7.4). This gives

$$(T^2 - 4n)S_q(T, n)^2 = 4b^2c > 0. \quad (7.9)$$

Thus $S_q(T, n) \neq 0$. Set $A = T/2$ and $B = b/S_q(T, n)$. The last identity yields $A^2 - cB^2 = n$. Therefore the numbers $\beta = A + B\sqrt{c}$ and $\beta' = A - B\sqrt{c}$ have trace T and norm n . Their q th powers have sum $D_q(T, n) = 2a$ and difference

$$(\beta - \beta')S_q(T, n) = 2b\sqrt{c}.$$

Consequently $\beta^q = a + b\sqrt{c}$, as required. $\square$

7.2 Consequences for implementation

The algorithm first asks whether the rational norm is a rational q th power, then finds rational roots of the degree- q trace polynomial. For $q = 3$, formula (7.8) is exactly the source's $T/2 + b\sqrt{c}/(T^2 - n)$. Testing the norm alone would be insufficient: the rational-trace root is also required.

The new path generates the two polynomials under an operation budget, extracts rational linear factors of $D_q(T, n) - 2a$, constructs (7.8), and checks the power identity before final acceptance. It is limited by the configured degree and root-index bounds. A rational-factor search is a construction mechanism inside the specified quadratic field; failure says nothing about representations in larger fields or by more general radicals.

For a negative real radicand, the real odd root is negative, whereas the principal complex fractional power has argument π/q . The implementation searches the positive magnitude and attaches the exact principal phase $(-1)^{1/q}$. A final exact gate remains appropriate even when a preceding field identity has been proved, because exponent numerators, alternate roots, and kernel expression transformations introduce additional branch choices.

8 Honsbeek's quartic: identity, recognition, and branches

Honsbeek's thesis studies the square root of a sum of two cube roots in a more specific field-theoretic setting. Theorem 104, Corollary 105, and formula (3.5) supply the relevant quartic construction [6]. For code review, the sufficient polynomial identity can be proved directly without asserting the full converse outside its hypotheses.

Let $A^3 = \alpha$ and $B^3 = \beta$ with $\alpha, \beta \in \mathbb{Q} \setminus \{0\}$, and define

$$F(s) = s^4 + 4s^3 + 8\frac{\beta}{\alpha}s - 4\frac{\beta}{\alpha}, \quad (8.1)$$

$$N = -\frac{s^2}{2}A^2 + sAB + B^2, \quad \Delta = \beta - s^3\alpha. \quad (8.2)$$

Proposition 8.1 (Honsbeek numerator identity). *For arbitrary complex choices of A and B with these cubes,*

$$N^2 - \Delta(A + B) = \frac{\alpha A}{4}F(s). \quad (8.3)$$

In particular, if $s \in \mathbb{Q}$ satisfies $F(s) = 0$, then $\Delta \neq 0$ and the two values $\pm N/\sqrt{\Delta}$ are the two square roots of $A + B$, with coincidence only when $A + B = 0$.

Proof. Expand N^2 . Replace A^3 by α and B^3 by β . After subtracting $(\beta - s^3\alpha)(A + B)$, the remaining terms are

$$A \left(\frac{s^4\alpha}{4} + s^3\alpha + 2s\beta - \beta \right),$$

which is (8.3). If $\Delta = 0$, then $\beta/\alpha = s^3$. The quartic equation becomes $9s^4 = 0$, forcing $s = 0$ and then $\beta = 0$, a contradiction. Since $(\sqrt{\Delta})^2 = \Delta$, division proves the stated square identity. The sign selecting the principal square root is still to be chosen. $\square$

8.1 F08: The recognizer is stricter than the sufficient identity

The original helper requires each of the two terms to have an explicit cube-root syntactic form, possibly multiplied by a rational coefficient, and requires $\Delta > 0$. This excludes a rational summand even though its cube is rational, and it rejects candidates with a negative denominator radicand before the final equality gate can select the correct complex phase. The prior report acknowledges the deliberately restricted recognizer [2].

A broader safe recognizer asks whether the cubes of the two exact terms reduce to nonzero rationals. Thus the family containing $\sqrt[3]{28} - 3$ is not excluded merely because -3 is written as a rational rather than as a cube root. For $\alpha = 28$, $\beta = -27$, $s = -3$, the quartic vanishes and $\Delta = 729$. The values must still be represented with the intended real cube-root summand; principal powers of a negative rational are not silently substituted for real cube roots.

The proposal allows $\Delta \neq 0$ instead of $\Delta > 0$ and offers both signs. This broadens candidate construction, not the acceptance rule. The full iff statement in the thesis has field and non-cube-ratio hypotheses; the implementation does not use this sufficient identity to claim a complete decision procedure for all two-term or higher-term nests.

8.2 A useful exact test family

Solving $F(s) = 0$ for the ratio gives

$$\frac{\beta}{\alpha} = \frac{s^3(s+4)}{4(1-2s)}, \quad s \neq 0, -4, \frac{1}{2}. \quad (8.4)$$

This generates exact parameterized checks of the numerator identity and denominator condition without relying on floating approximations. The verification script uses such parameter choices as well as the explicit benchmark cases. These verify the algebra behind candidate construction; they do not determine how a native kernel chooses to print or simplify the resulting expression.

9 Square-class cosets: a concrete gap and a bounded repair

9.1 F07: Neither of the two current bases covers all elementary cases

The existing multi-surd helper builds two possible rational spans: one generated by 1 and the square roots of the primes in the input radicands, and another that also includes the input square roots. The prior analysis explicitly identifies this as heuristic rather than complete [1, 2]. The following example

makes the missing geometry precise:

$$\gamma = \sqrt{6} + \sqrt{35} + \sqrt{77}, \quad (9.1)$$

$$\rho = \gamma^2 = 118 + 2\sqrt{210} + 14\sqrt{55} + 2\sqrt{462}. \quad (9.2)$$

All terms of γ are positive, so $\sqrt{\rho} = \gamma$. The source's prime set is $\{2, 3, 5, 7, 11\}$, and its input radicands are 55, 210, 462. Neither proposed basis contains 6, 35, 77. Distinct squarefree monomials are linearly independent over $\mathbb{Q}$ in the ambient multiquadratic field, so neither span can contain γ or $-\gamma$. Since these are the only square roots of ρ in that field, the two particular rational systems cannot produce the desired root. Another part of the public denester may still discover it; this is a proved limitation of the helper's search space.

9.2 Closing only the input classes still does not suffice

Let

$$M = \mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{5}, \sqrt{7}, \sqrt{11}).$$

Represent a squarefree radicand by its prime-exponent vector over $\mathbf{F}_2$. The subgroup generated by the input classes is

$$H = \{1, 55, 210, 462\}. \quad (9.3)$$

It has rank two, so the corresponding field K has degree four. The ambient field M has degree 32. The root γ lies not in the span of H , but in the coset

$$6H = \{6, 35, 77, 330\}. \quad (9.4)$$

A repair that merely computes closure of the input radicands would still miss this example.

Theorem 9.1 (Single-coset support). *Let $M/\mathbb{Q}$ be a multiquadratic extension with square-class monomial basis, and let K correspond to a subgroup H of those square classes. If $0 \neq \gamma \in M$ satisfies $\gamma^2 \in K$, then the support of γ in the monomial basis is contained in one coset of H .*

Proof. For each $\sigma \in \text{Gal}(M/K)$, the equality $(\sigma\gamma)^2 = \gamma^2$ implies $\sigma\gamma = \pm\gamma$. Because $\gamma \neq 0$, the sign is well defined and gives a character χ of $\text{Gal}(M/K)$. Each basis monomial is itself an eigenvector for this group, with a character determined by its square class. Comparing coefficients in the linearly independent monomial basis shows that every monomial with nonzero coefficient in γ restricts to the character χ .

Two square-class characters have the same restriction to $\text{Gal}(M/K)$ precisely when their quotient is a character belonging to K , equivalently when their square classes differ by an element of H . Therefore all supporting classes lie in the same coset of H . $\square$

9.3 An explicit four-variable system

Write a candidate in the appropriate coset as

$$x_1\sqrt{6} + x_2\sqrt{35} + x_3\sqrt{77} + x_4\sqrt{330}, \quad x_i \in \mathbb{Q}.$$

The square equals (9.2) exactly when

$$6x_1^2 + 35x_2^2 + 77x_3^2 + 330x_4^2 = 118, \quad (9.5)$$

$$12x_1x_4 + 14x_2x_3 = 14, \quad (9.6)$$

$$2x_1x_2 + 22x_3x_4 = 2, \quad (9.7)$$

$$2x_1x_3 + 10x_2x_4 = 2. \quad (9.8)$$

The coefficients $(1, 1, 1, 0)$ solve the system. Their negative gives the other square root. All coefficients and equations are rational; a native symbolic solver need not reason about nested radicals merely to build this system.

9.4 Coefficient arithmetic without expansion of radical expressions

For positive squarefree integers d_i, d_j , put $g = \gcd(d_i, d_j)$. Then

$$\sqrt{d_i}\sqrt{d_j} = g\sqrt{d_id_j/g^2}. \quad (9.9)$$

The new implementation stores the coefficient of each square class directly, using the integer GCD for the rational factor and bitwise XOR for the square class. This avoids expanding a large symbolic square and subsequently extracting radical coefficients. The independent verification exhausts the 32×32 multiplication table for the example's ambient basis.

The optional coset search retains the two inexpensive original bases first. If they fail, it enumerates bounded coset bases and solves the corresponding rational systems. Its defaults cap the ambient prime rank at six, the basis size at sixteen, and the number of cosets at sixteen. Exceeding those limits records a search limit; it is not a proof of non-denestability.

9.5 The exact completeness boundary

Theorem 9.1 is complete *inside a specified ambient multiquadratic field* when all cosets are considered and their rational systems are fully solved. The implemented routine is a clipped search with a general-purpose solver, so it makes no such unconditional completeness claim. Furthermore, a root may require a prime class outside the selected universe, an indirect fourth-root representation, complex generators, or a nonmultiquadratic extension. Neither a solved collection of local systems nor exhaustion of this particular prime universe answers the unrestricted radical-denesting problem.

An additional implementation limit is that the recognizer expects a rational combination of positive square roots with integral radicands after ordinary evaluation and expansion. It does not implement a full algebraic normalization of arbitrary equivalent presentations before entering this fast path.

10 Multiplier search, polynomial GCDs, and rationalization

10.1 Why a multiplier can help

The general backend chooses an index q associated with the deepest radical nodes and studies a suitable power of the target. Multipliers modify the auxiliary algebraic number before minimal-polynomial and polynomial-GCD calculations. A favorable multiplier may expose a small-degree relation over the chosen coefficient field even when the original presentation does not. Low-degree roots are reconstructed into radicals, divided by a selected multiplier root, and tried with roots of unity [1].

The field specified by all radicals visibly occurring in an expression can be larger than the field generated by that expression's value. Thus an informal comment saying “over $\mathbb{Q}(\rho)$ ” should not be used as an exact description of an Extension list made from the radicals in ρ . Searching a larger explicit field may produce more candidates; the final exact check, rather than the informal field description, is the safety boundary.

Proposition 10.1 (Phase-complete reconstruction). *Let $E, m \in \mathbb{C} \setminus \{0\}$, let $q \geq 2$, put $\rho = E^q$, and suppose $z^q = m\rho$. For any selected q th root μ of m , the value E is among*

$$z\mu^{-1}\zeta_q^k, \quad 0 \leq k < q, \quad (10.1)$$

where $\zeta_q = e^{2\pi i/q}$.

Proof. $(z/(\mu E))^q = 1$, so the quotient is a q th root of unity. Multiplication by the reciprocal of that root supplies E in the displayed orbit. $\square$

This proof justifies a candidate orbit, not unconditional distribution of fractional powers across products. Zero multipliers must be excluded. In a bounded implementation it may be prudent to canonicalize and certify nonzeroness only when a multiplier is selected for actual use, rather than imposing expensive normalization on every speculative queue element.

10.2 Horner rationalization is mathematically justified

If a nonzero algebraic denominator d satisfies

$$p(d) = a_0 + a_1d + \cdots + a_nd^n = 0, \quad a_0 \neq 0,$$

then

$$\frac{1}{d} = -\frac{a_1 + a_2d + \cdots + a_nd^{n-1}}{a_0}. \quad (10.2)$$

This follows by moving the constant term to the other side and dividing by a_0d . Horner evaluation produces the numerator using $n - 1$ multiplications instead of explicitly constructing all powers. The source's constant-term guard and subsequent identity check are appropriate. A minimal polynomial is one way to obtain the relation, but minimality is not needed for (10.2); any valid annihilating polynomial with nonzero constant term suffices.

Rationalization itself need not decrease radical depth or the overall cost. It is correctly treated as a candidate-generation or polishing operation. A standalone rationalization helper should promise equality and a rationalized denominator when successful, not unconditional denesting.

10.3 F12: Queue keys and local batch limits need precise descriptions

A string key derived from an expanded expression is a syntactic normalization, not a canonical algebraic-number key. Different formulas for the same multiplier can still be admitted separately. This loses search capacity rather than proving an unsound result. The proposal uses expression keys without string serialization and offers an optional exact canonicalization mode. Canonicalization itself can be expensive, so it remains disabled by default.

The discriminant-derived proposal code also has an inner prime-power loop whose nominal small batch quota is checked only at a coarser level. For a sufficiently large root index, one inner loop can exceed that local quota even though the centralized total admission and proposal limits remain intact. The derivative checks the batch limit inside the relevant inner loop. This is a minor bounded-work discrepancy, not a claim that the already centralized queue grows without bound.

A promising larger redesign is to search equivalence classes of multipliers modulo q th powers, since many such multipliers induce equivalent reconstruction problems. That requires careful treatment of coefficient fields and branches and is not implemented merely by renaming a syntactic key “canonical”.

11 Budgets, traversal, costs, and observability

11.1 F06: Resource controls need a narrower and more accurate promise

The original has a shared deadline and operation wrappers, which are major improvements. However, some `FactorInteger` calls, coefficient expansions, and related preparation steps are not individually wrapped, and some preflight or final cost computations occur outside the outer protected search. A degree limit checked after `MinimalPolynomial` returns restricts subsequent use of that polynomial; it does not limit the cost already spent obtaining it [1].

The proposal places preflight and cost computation inside the core session and wraps the identified heavy algebraic operations. Known-degree auxiliary polynomials such as $x^k - \rho$ can be rejected before construction when their index already exceeds a bound. A final report may contain `Missing["NotComputed"]` for a cost after resource exhaustion instead of performing fresh unbounded work after the deadline.

The contract still must not promise a hard operating-system deadline. Wolfram's time and memory controls are cooperative, and protected computation can affect abort behavior. `MemoryConstrained` measures additional memory requested during the evaluation, not the total resident size of the process or an already allocated input [13, 14]. Argument and option evaluation occurs before these public functions establish the session; serialization and external callback side effects are not covered. Hard containment would require a separately supervised kernel process with external time and memory enforcement.

Operation deadlines and the global deadline also answer different questions. An expensive candidate may time out locally while ample total time remains. The report should say so, rather than treating every local failure as global timeout or treating a cooperative deadline check as an ordinary unsuccessful mathematical search.

11.2 F10: The cost is not compositional

Consider the positive identity

$$A = \sqrt{\frac{3}{2} + \sqrt{3}} = 3^{1/4} \frac{1 + \sqrt{3}}{2} = B. \quad (11.1)$$

The right-hand side is a flat radical expression, while the left has depth two. It can have a larger expression size. If this entry is placed beside an unrelated depth-three entry in a list, lowering its depth does not lower the list's maximum depth. Radical-node and leaf counts can then defeat a global lexicographic cost decrease. A local success may consequently be rolled back.

This behavior is explicitly acknowledged by the prior report. It is not a value-correctness bug and should not be "fixed" by silently changing the cost contract. The proposal preserves the global default but offers `"ListProgress" -> "Independent"`: separate list entries may each improve under their own cost. This is useful for lists of independent problems, while deliberately allowing a composite cost that would not have passed the global rule. Sums and products continue to use the stricter reconstruction policy.

A more general solution would retain a bounded Pareto set of candidates measured by depth, node count, size, and representation type, and choose among them after reconstruction. That is a search redesign, not a small comparator correction. It remains future work.

11.3 F11: Mixed inexact input is a policy choice, not exact-island traversal

The current implementation conservatively leaves the entire host unchanged when it contains an inexact number. That avoids rationalizing numerical input or obscuring floating-point provenance, but it also prevents an independent exact radical elsewhere in a list from being denested. The prior review documents this choice [2].

The new "ProcessExactIslands" -> True mode permits traversal of mixed hosts while still replacing only subtrees accepted by the exact grammar. Approximate leaves are not rationalized. The default remains the conservative original policy. Neither mode claims to traverse arbitrary Wolfram expressions: lists, ordinary sums, products, and rational powers are the intended hosts. Hold, arbitrary functions, associations, and user-defined evaluation semantics are outside this congruence-based traversal contract.

11.4 F13: Candidate evidence is not a final-result proof chain

A search can accept a candidate and later replace it with a better candidate, or reject the enclosing reconstruction under the global cost. A record of all accepted candidates is therefore not automatically an ancestry chain leading to the final output. The prior analysis correctly notes that such records are not independently checkable proof objects; the remaining interface improvement is to make that fact machine-readable.

The proposal exposes accepted-search-candidate records separately from committed-pass records, includes explicit scope labels, and marks trace or record truncation. A commit is applied before its journal entry is appended; if an interruption occurs in between, the report marks the commit journal as potentially incomplete. No transactionality across arbitrary kernel interrupts is claimed. "ResultChanged" is separate from termination status, so a run may return a certified improvement and still report that it later exhausted its time budget.

A future independent certificate format would include algebraic generators and relations, a representation of each candidate in that field, a polynomial identity or zero-reduction certificate, and isolating/branch data sufficient to select the intended conjugate. Recording only `RootReduce[candidate-target]==0` is reproducible kernel evidence, not a kernel-independent proof.

11.5 F14: Argument validation should be reached consistently

An `OptionsPattern` definition may not match a malformed call such as `Strad[e, 17]`, so the internal validator never gets the opportunity to return its structured `Failure`. The derivative routes public argument sequences through one resolver, recognizes the optional Boolean compatibility flag explicitly, validates rule-shaped options, and gives a structured missing-input result. Defaults are resolved from the actual public head, preserving separately configured `DenestRadicals` or `DenestReport` defaults.

This is an interface consistency improvement, not a mathematical repair. Option evaluation itself may execute user-supplied code before the session starts; a robust package should document that limitation rather than imply that validating the resulting option values establishes a security boundary.

12 The proposed StradFixed3.wl

12.1 Packaging and compatibility

The delivered file is self-contained and loads into `RadicalDenest3'` rather than overwriting `RadicalDenest2'`. It preserves the principal public names and existing string-option style, retains the cubic/quadratic, Honsbeek, linear-factor, polynomial-GCD, and multiplier architecture, and incorporates the changes above. It is a proposed derivative, not a drop-in production release with demonstrated regression parity.

The separate context allows comparison without modifying the upstream package:

```
Get["path/to/StradFixed2.wl"];
Get["code/StradFixed3.wl"];
a = RadicalDenest2'Strad[expr];
b = RadicalDenest3'Strad[expr];
```

`Global ClearAll["Global '*"]` operations are not needed. Session memoization is dynamically scoped, so results and resource-dependent state do not persist across unrelated calls.

12.2 Important default-preserving and opt-in choices

Option	Default	Meaning
"HigherOddQuadratic"	True	Trace-norm construction for bounded odd indices beyond three.
"SquareClassSearch"	False	Try cosets after the inexpensive multi-surd bases.
"MaxPrimeRank"	6	Maximum ambient prime rank for coset enumeration.
"MaxBasisSize"	16	Maximum number of coefficients in a surd system.
"MaxCosets"	16	Maximum admitted coset systems.
"CanonicalMultipliers"	False	Pay for exact canonicalization of multiplier keys.
"ProcessExactIslands"	False	Permit exact rewrites within mixed exact/inexact hosts.
"ListProgress"	"Global"	Optional "Independent" treats list entries separately.

The established search limits remain explicit: trial allowance, global and per-operation times, certificate time, additional-memory allowance, multiplier cap, root index, degree, solve degree, leaf count, pass count, recursion, patience, and trace length. A `"TimeBudget"` of zero disables search. The standalone equality/type helpers use a short bounded session based on the main defaults; they are not unlimited theorem provers.

12.3 Examples and expected obligations

The following calls are reproducible instructions, not a transcript of native results from this review:

```

Get["code/StradFixed3.wl"];
RadicalDenest3'Strad[Sqrt[5 + 2 Sqrt[6]]]
RadicalDenest3'EqualityStatus[Sqrt[2], -Sqrt[2]]
RadicalDenest3'ExactAlgebraicQ[Root[#^3 + Pi # + 1 &, 1]]

rho = 118 + 2 Sqrt[210] + 14 Sqrt[55] + 2 Sqrt[462];
RadicalDenest3'DenestReport[Sqrt[rho],
  "SquareClassSearch" -> True, "NumericPrefilter" -> False]

RadicalDenest3'Strad[{0.5, Sqrt[5 + 2 Sqrt[6]]},
  "ProcessExactIslands" -> True, "ListProgress" -> "Independent"]

```

The first example should produce a certified cheaper expression, not necessarily a fixed pretty-printed ordering. The equality test must distinguish conjugates. The root with π coefficient must not be classified as supported algebraic input. The coset example should reach the mathematically exhibited root when the bounded solver completes. The mixed-input example must not convert 0.5 into a rational exact number.

12.4 Remaining design limitations of the proposal

The derivative is still heuristic and representation-sensitive. Exact algebraic operations may be expensive; an operation timeout may prevent a true equality from being established. General-purpose `Solve` may leave a rational system unresolved [15]. Positive-only caching can repeat unsuccessful work. Enumerating all phase choices can be costly. The optional coset path does not cover arbitrary fourth roots or larger field extensions. A strict type recognizer can conservatively reject valid but unsupported algebraic presentations. The inherited maximum-depth objective can suppress useful local changes in non-list contexts.

Most importantly, static review and the supplied mathematical tests do not establish that this source loads and behaves correctly on the user's target kernel. The native suite is part of the deliverable precisely because package syntax, evaluation semantics, and actual solver behavior require that final check.

13 Verification performed and the native release gate

13.1 Executed independent checks

The archive's `verification/verify_math.py` was executed with Python 3.13.5 and SymPy 1.14.0. Its machine-readable `results.json` records the exact package hash and the actual result. The final run passed 3,412 assertions, separated as follows.

Family	Assertions	Evidence type
Dickson polynomial identities	5	Exact symbolic
Odd-root parameter checks	1,280	Exact rational-field
Direct quadratic checks	144	Exact identities
Indirect quadratic checks	270	Exact identities
Gaussian checks	80	Exact identities
Honsbeek checks	48	Exact identities
Square-class example checks	9	Exact algebra

Family	Assertions	Evidence type
Square-class multiplication table	1,024	Exact arithmetic
Polynomial inverse checks	6	Exact polynomial remainders
Principal-branch checks	6	Exact selected examples
Roots-of-unity arithmetic	527	Exact phase arithmetic
Mathematical subtotal	3,399	Not native package tests
Control-flow model	3	Python model only
Lexical checks	5	Delimiters, comments, strings
Structural checks	5	Source-pattern checks
Total	3,412	All passed

The odd-index checks cover 320 parameter cases at indices 3, 5, 7, 9, with separate symbolic identities also tested at index 11. There are 72 direct-quadratic, 90 indirect-quadratic, 40 Gaussian, 22 Honsbeek-parameter, and six polynomial-inverse cases. Multiple assertions may belong to the same case; these counts must not be advertised as thousands of independent end-to-end denesting examples.

The phase arithmetic and multiplication-table checks include elementary identities. Their value is reproducibility and coverage of indexing conventions, not statistical evidence of package reliability. The three control-flow assertions model the incumbent and negative-cache logic in Python rather than execute Wolfram definitions. Lexical balance is not a Wolfram parser and cannot establish correct evaluation. An early lexical pass did catch missing brackets in a draft of the proposal; the delivered file passes those checks, but that is still not a substitute for native loading.

13.2 Native tests supplied but not run

The 52 `VerificationTest` cases in `tests/StradFixed3.wlt` cover the public API and options, exact grammar and nonalgebraic-root rejection, equality and conjugates, selected denesting identities and branches, timeout and trace behavior, incumbent preservation, index reduction, cache behavior, mixed hosts, and optional square-class construction. Additional scripts probe the original implementation and compare selected old/new cases.

The connected Wolfram service returned an HTTP 404 error at its MCP endpoint before even the diagnostic evaluation could execute. No local Wolfram kernel was available. Consequently there is no native pass count, no measured speedup, no demonstrated message-free loading, and no native regression-parity claim for `StradFixed3.wl` in this article.

13.3 A concrete release gate

Run the native package suite in a clean kernel first, followed by the original-source probes and the differential script. Then rerun the repository's existing 144 tests, its 116-case battery, and its structured randomized cases against the derivative. Keep separate columns for exact equality, cost/depth progress, elapsed time, messages, and termination limits. A change that denests more examples but increases false "Different" statuses or silently exceeds the intended resource profile is not an unqualified improvement.

For the index and incumbent findings, test helper contracts as well as public outputs: public recovery by a different stage can hide the internal regression. For negative caching, deliberately control the recursion/trial profile instead of relying on a single naturally hard expression. For the new coset search, test both the explicit successful coset and a clipped configuration that must report an incomplete search. For external callbacks, test wrong numerical proposals under symbolic hosts; equality must be certified at the replacement boundary.

A final production benchmark should include large coefficient heights and algebraic degrees, not only many low-degree examples. Degree, coefficient bit size, number of generators, expression size, and branch complexity contribute differently to runtime. Total time alone can conceal which change is responsible for a regression.

14 Prioritized development directions

The immediate priority is to preserve the local acceptance architecture while repairing the domain predicate and search handoffs. These are relatively small conceptual changes with clear contracts. Next, separate exact decisions from heuristic search rejection and make resource/termination outcomes explicit. Those changes improve the meaning of the API even when they do not denest one additional example.

The odd-quadratic and square-class-coset constructions offer targeted coverage gains with short mathematical explanations. Their resource use should be measured before broadening defaults. For multiquadratic problems, a canonical square-class basis and linear algebra over $\mathbf{F}_2$ should eventually become shared infrastructure rather than a specialized helper. For general multiplier problems, classes modulo powers, reusable number-field embeddings, and operation caches offer more principled search reduction than syntactic queue deduplication.

A bounded Pareto frontier could improve compositional simplification, but its acceptance and termination invariants should be specified before implementation. Independent certificates are another valuable but separate project: they require explicit field and embedding data, not just a larger trace. Finally, reproducible performance claims require a pinned source commit, a recorded kernel/platform version, fixed input hashes, and the complete machine-readable run results.

15 Conclusion

The second repaired denester is not merely the original heuristic wrapped in a superficial equality check. It has local acceptance, explicit budgets, a richer collection of mathematical fast paths, and a substantially more disciplined public boundary. The audit should preserve those strengths.

The main remaining source-level concerns are that exact-looking opaque objects are trusted too broadly, search stages can forget a better incumbent, useful index reductions are withheld until recursion changes them, and negative memo entries can encode the accident of an earlier resource budget. Other concerns—mixed inexact hosts, global cost rollback, numerical rejection, limited bases, and non-independent certificates—are largely explicit design choices whose contracts can be made clearer or whose coverage can be extended.

The supplied derivative embodies these changes and includes two proved construction principles: odd-index trace–norm reconstruction in a real quadratic field and single-coset support for square roots in a multiquadratic ambient field. The independent exact checks support the mathematics. Native package correctness and performance remain the explicitly identified release gate, not an inference drawn from those mathematical checks.

A Reproduction commands and archive map

Compile the article from the archive root so the code listing can be found:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
python verification/verify_math.py
wolframscript -file tests/run_tests.wls
wolframscript -file tests/probe_original.wls /path/to/StradFixed2.wl
wolframscript -file tests/differential.wls /path/to/StradFixed2.wl
```

The Python script requires SymPy. The native scripts require a licensed or otherwise available Wolfram kernel and should be run with a working directory at the archive root. Their actual command-line handling is documented in the files and README. No external package source is downloaded automatically, and the original repository is not modified.

The archive contains the article source and PDF, the self-contained proposed package, native tests and runners, the independent verification script and its actual results, a source-access ledger, and hashes. It does not bundle third-party papers, font files, an unverified upstream source copy, or invented native test logs.

B Selected regression obligations

Boundary	Required observation on a native run
Algebraic grammar	Rational-polynomial roots accepted; roots with π coefficients or free parameters not admitted merely by head and precision.
Equality	Equal radicals certified; distinct conjugates certified different without depending on a numerical filter.
Incumbent	An inapplicable or unsuccessful later stage returns the supplied better incumbent.
Index reduction	The raw candidate in (5.1) is eligible even with recursion disabled.
Memoization	Failure under a reduced resource profile does not become a persistent negative certificate.
Square classes	The coset in (9.2) is considered when enabled and not silently claimed exhausted when clipped.
Principal branches	Negative real radicands and complex phases preserve the selected principal value.
Symbolic hosts	A wrong callback proposal for a numerical subtree is rejected locally.
Mixed hosts	Opt-in exact-island processing leaves approximate leaves approximate.
Resource reporting	Local operation timeout, global timeout, unchanged result, and changed result are distinguishable.
Logs	Accepted-candidate records and committed-pass records are not conflated; truncation is visible.
Compatibility	The old test batteries are rerun rather than inherited by assertion.

References

- [1] Vladimir Reshetnikov, RadicalDenest repository, *StradFixed2.wl*, current main source inspected September 5, 2026. <https://github.com/VladimirReshetnikov/RadicalDenest/blob/main/src/corrected/StradFixed2.wl>.
- [2] RadicalDenest repository, *Unified analysis B*, current review inspected September 5, 2026. Repository-reported native results are distinguished from this audit's checks. <https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/src/code-review/unified-B>.
- [3] RadicalDenest repository, *Radical Denesting: Unified Research Guide*, current report and literature collection. <https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/docs/report>; <https://github.com/VladimirReshetnikov/RadicalDenest/tree/main/docs/literature>.
- [4] Allan Borodin, Ronald Fagin, John E. Hopcroft, and Martin Tompa, "Decreasing the Nesting Depth of Expressions Involving Square Roots," *Journal of Symbolic Computation* 1 (1985), 169–188. <https://www.cs.toronto.edu/~bor/Papers/decreasing-nesting-depth-for-expressions.pdf>.
- [5] Eleftherios Gkioulekas, "On the denesting of nested square roots," *International Journal of Mathematical Education in Science and Technology* 48(6) (2017), 942–953. Author manuscript dated December 21, 2016. <https://faculty.utrgv.edu/eleftherios.gkioulekas/papers/submitted/denesting-square-roots.pdf>.
- [6] M. Honsbeek, *Radical Extensions and Galois Groups*, Ph.D. thesis, Radboud University Nijmegen, 2005. Chapter 3, Theorem 104, Corollary 105, and formula (3.5). https://www.math.ru.nl/~bosma/students/honsbeek/M_Honsbeek_thesis.pdf.
- [7] David J. Jeffrey and Albert D. Rich, "Simplifying Square Roots of Square Roots by Denesting," in M. J. Wester (ed.), *Computer Algebra Systems: A Practical Guide*, Wiley, 1999, 61–72. Accessible manuscript: <https://cybertester.com/data/denest.pdf>.
- [8] Wolfram Research, *Root*, official Wolfram Language documentation, accessed September 5, 2026. Includes nonalgebraic numeric coefficients and symbolic parameters. <https://reference.wolfram.com/language/ref/Root.html>.
- [9] Wolfram Research, *PossibleZeroQ*, official documentation, especially the "ExactAlgebraics" method and the limitations of approximate testing. <https://reference.wolfram.com/language/ref/PossibleZeroQ.html>.
- [10] Wolfram Research, *RootReduce*, official documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.
- [11] Wolfram Research, *Power*, official documentation, principal complex values. <https://reference.wolfram.com/language/ref/Power.html>.
- [12] Wolfram Research, *Order*, official documentation, sign convention for canonical order. <https://reference.wolfram.com/language/ref/Order.html>.
- [13] Wolfram Research, *TimeConstrained*, official documentation, resource and abort semantics. <https://reference.wolfram.com/language/ref/TimeConstrained.html>.
- [14] Wolfram Research, *MemoryConstrained*, official documentation, additional-memory and abort semantics. <https://reference.wolfram.com/language/ref/MemoryConstrained.html>.
- [15] Wolfram Research, *Solve*, official documentation, domains, rational constraints, and generic solution behavior. <https://reference.wolfram.com/language/ref/Solve.html>.

C Complete proposed implementation

The listing is the delivered code/StradFixed3.wl, included verbatim from that file during compilation. Its native execution status is as stated in Section 13. The standalone source file is the preferred form for running or editing the package.

```

1  (* ::Package:: *)
2  (* StradFixed3.wl -- proposed derivative of StradFixed2.wl, 2026-09-05.
3     Based on the RadicalDenest repository's second corrected implementation.
4     This file retains its principal algorithms, but is NOT a byte-for-byte patch.
5     Native Wolfram execution was unavailable during preparation. Run the supplied
6     tests and the repository's unified-B battery before adopting this proposal.
7
8     Trust boundary: a replacement must be an explicit exact radical expression,
9     strictly cheaper, and exactly equal to the selected input value. Numerical
10    rejection is optional and NEVER constitutes an exact "Different" certificate.
11    Root and AlgebraicNumber payloads are validated, not trusted by their heads.
12    Budgets cover the core session, not prior argument/option evaluation, callback
13    side effects, or serialization. Kernel resource controls are cooperative.
14    No completeness, optimal-depth, or universal performance claim is made.
15 *)
16 BeginPackage["RadicalDenest3"];
17 Strad::usage = "Strad[expr, opts] tries certified, cost-decreasing radical rewrites. Strad[expr, True, opts]
    enables inner levels and bounded repeated passes.";
18 DenestRadicals::usage = "DenestRadicals[expr, opts] is the option-driven entry point, with its own defaults.";
19 DenestCore::usage = "DenestCore[expr, opts] processes one exact algebraic number.";
20 DenestReport::usage = "DenestReport[expr, opts] returns the result, termination information, accepted search
    candidates, and committed passes. Candidate records are not standalone proof objects.";
21 EqualityStatus::usage = "EqualityStatus[a,b] is Equal, Different, or Unknown (as strings). Equal and Different
    require exact algebraic evidence; resource exhaustion or an unsupported representation gives Unknown.";
22 CertifiedEqualQ::usage = "CertifiedEqualQ[a,b] is True only for an exact Equal certificate.";
23 ExactAlgebraicQ::usage = "ExactAlgebraicQ[e] recognizes the supported exact algebraic grammar. Opaque payloads
    require a rational defining polynomial or certified canonicalization. False may mean unsupported or out of
    budget.";
24 RadicalExpressionQ::usage = "RadicalExpressionQ[e] recognizes finite expressions made from Gaussian rationals using
    Plus, Times, and rational Power; no opaque algebraic objects.";
25 RadicalDepth::usage = "RadicalDepth[e] counts nested nonintegral rational powers, treating opaque algebraic objects
    as atoms.";
26 RadicalCost::usage = "RadicalCost[e] is {opaque-object count, maximum radical depth, radical-node count, LeafCount,
    rational bit size}.";
27 RationalizeDenominator::usage = "RationalizeDenominator[e] uses a certified polynomial inverse of the exact
    algebraic denominator. It need not lower RadicalCost.";
28 Factorc::usage = "Factorc[e] offers Factor[e] through the exact acceptance gate; symbolic input is unchanged.";
29 Options[Strad] = {
30   "AllLevels" -> False, "Verbose" -> False, "Trace" -> False,
31   "Multipliers" -> Automatic, "Solver" -> Automatic, "Factor" -> True,
32   "MaxTrials" -> 120, "TimeBudget" -> 120, "OperationTime" -> 30,
33   "CertifyTime" -> 20, "MemoryBudget" -> 1073741824,
34   "MultiplierCap" -> 1000, "MaxRootIndex" -> 32, "MaxDegree" -> 64,
35   "MaxSolveDegree" -> 4, "MaxLeafCount" -> 20000, "MaxPasses" -> 4,
36   "MaxRecursion" -> 3, "Patience" -> 25, "NumericPrefilter" -> True,
37   "MaxTraceEntries" -> 200,
38   "HigherOddQuadratic" -> True, "SquareClassSearch" -> False, "MaxPrimeRank" -> 6,
39   "MaxBasisSize" -> 16, "MaxCosets" -> 16,
40   "CanonicalMultipliers" -> False, "ProcessExactIslands" -> False,
41   "ListProgress" -> "Global"
42 };
43 Options[DenestRadicals] = Options[Strad];
44 Options[DenestCore] = Options[Strad];
45 Options[DenestReport] = Options[Strad];
46 Begin["Private"];
47
48 (* Session state is dynamically scoped. No persistent problem/result cache. *)
49 $active = False; $cfg = Association[Options[Strad]]; $deadline = Infinity;
50 $stats = <||>; $limits = <||>; $trace = {}; $records = {}; $commits = {};
51 $recordsTruncated = False; $traceTruncated = False; $commitsTruncated = False;
52 $memo = <||>; $inProgress = <||>; $typeMemo = <||>; $seqMemo = <||>;
53 $trialUse = <||>; $recursion = 0; $x = Unique["RadicalDenest3Private"x"];

```

```

54 newStats[] := Association[Thread[{
55   "Islands", "Trials", "Operations", "OperationTimeouts", "OperationFailures",
56   "Certificates", "CertificatesEqual", "CertificatesDifferent", "CertificatesUnknown",
57   "CertificateCacheHits", "NumericRejections", "CandidatesAccepted", "MultipliersProposed",
58   "MultipliersAdmitted", "DuplicateMultipliers", "FastPathAccepted", "PassesCompleted",
59   "SquareClassSystems", "PositiveMemoHits"} -> ConstantArray[0, 19]]];
60 bump[k_String] := AssociateTo[$stats, k -> (Lookup[$stats, k, 0] + 1)];
61 limitHit[k_String] := AssociateTo[$limits, k -> True];
62 remaining[] := $deadline - AbsoluteTime[];
63 expiredQ[] := If[remaining[] <= 0, limitHit["TimeBudget"]; True, False];
64 log[a_] := If[TrueQ[$cfg["Verbose"]], Print["RadicalDenest3 ", a]];
65 trace[tag_String, data_:<|>] := If[TrueQ[$cfg["Trace"]],
66   If[Length[$trace] < $cfg["MaxTraceEntries"],
67     AppendTo[$trace, <|"Event" -> tag, "Data" -> data|>], $traceTruncated = True]];
68 SetAttributes[bounded, HoldAll];
69 bounded[body_, kind_: "Operation"] := Module[{seconds, value},
70   seconds = Min[remaining[], If[kind === "Certificate", $cfg["CertifyTime"], $cfg["OperationTime"]]];
71   If[! TrueQ[seconds > 0], limitHit[If[remaining[] <= 0, "TimeBudget", kind <> "Time"]]; Return[$Failed]];
72   bump["Operations"];
73   value = Quiet[Check[TimeConstrained[body, seconds, operationTimedOut], operationFailed]];
74   Which[value === operationTimedOut,
75     bump["OperationTimeouts"]; limitHit[If[kind === "Certificate", "CertifyTime", "OperationTime"]];
76     If[remaining[] <= 0, limitHit["TimeBudget"]]; $Failed,
77     MemberQ[{operationFailed, $Failed, $Aborted}, value], bump["OperationFailures"]; $Failed,
78     True, value]];
79
80 (* Options are resolved once using the defaults of the actual public head. *)
81 rationalQ[e_] := MatchQ[e, _Integer | _Rational];
82 finiteNonnegativeQ[v_] := MatchQ[v, _Integer | _Rational | _Real] && TrueQ[0 <= v < Infinity];
83 booleanQ[v_] := v === True || v === False;
84 flattenRules[items_List] := Flatten[Replace[items, l_List -> flattenRules[l], {1}], 1];
85 resolveOptions[head_Symbol, raw_List] := Module[{rules, names, unknown, cfg, bad},
86   rules = flattenRules[raw];
87   If[! AllTrue[rules, MatchQ[#, _Rule | _RuleDelayed] &],
88     Return[Failure["InvalidOption", <|"MessageTemplate" -> "Options must be rules."|>]];
89   names = First /@ Options[head]; unknown = Complement[First /@ rules, names];
90   If[unknown != {}, Return[Failure["UnknownOption", <|"Keys" -> unknown|>]]];
91   cfg = Association[Table[With[{name = key}, name -> OptionValue[head, rules, name]], {key, names}]];
92   bad = Join[
93     Select[{"AllLevels", "Verbose", "Trace", "Factor", "NumericPrefilter", "SquareClassSearch",
94       "CanonicalMultipliers", "ProcessExactIslands", "HigherOddQuadratic"}, ! booleanQ[cfg[#] &]],
95     Select[{"MaxTrials", "MultiplierCap", "MaxTraceEntries", "MaxRecursion", "Patience"},
96       ! (IntegerQ[cfg[#] && cfg[#] >= 0) &],
97     Select[{"MemoryBudget", "MaxRootIndex", "MaxDegree", "MaxSolveDegree", "MaxLeafCount",
98       "MaxPasses", "MaxPrimeRank", "MaxBasisSize", "MaxCosets"},
99       ! (IntegerQ[cfg[#] && cfg[#] >= 1) &],
100     Select[{"TimeBudget", "OperationTime", "CertifyTime"}, ! finiteNonnegativeQ[cfg[#] &]];
101   If[! (cfg["Multipliers"] === Automatic || ListQ[cfg["Multipliers"]]), AppendTo[bad, "Multipliers"]];
102   If[! (cfg["Solver"] === Automatic || MatchQ[cfg["Solver"], _Function | _Symbol]), AppendTo[bad, "Solver"]];
103   If[cfg["MaxSolveDegree"] > 4, AppendTo[bad, "MaxSolveDegree"]];
104   If[! MemberQ[{"Global", "Independent"}, cfg["ListProgress"]], AppendTo[bad, "ListProgress"]];
105   If[bad != {}, Failure["InvalidOption", <|"Keys" -> DeleteDuplicates[bad]|>], cfg]];
106
107 (* A Root head and infinite precision are NOT an algebraicity certificate. *)
108 gaussianQ[e_] := rationalQ[e] || (Head[e] === Complex && rationalQ[Re[e]] && rationalQ[Im[e]]);
109 opaqueQ[e_] := MatchQ[e, _Root | _AlgebraicNumber];
110 RadicalExpressionQ[e_] := Which[
111   gaussianQ[e], True, AtomQ[e], False,
112   MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, RadicalExpressionQ],
113   Head[e] === Power && Length[e] === 2, rationalQ[e[[2]]] && RadicalExpressionQ[e[[1]]],
114   True, False];
115 rationalPolynomialQ[p_, x_] := FreeQ[p, $Failed | $Aborted | operationTimedOut | operationFailed] &&
116   TrueQ[PolynomialQ[p, x]] && IntegerQ[Exponent[p, x]] && Exponent[p, x] >= 1 &&
117   AllTrue[CoefficientList[p, x], rationalQ];
118 ordinaryRootQ[r_] := Module[{p, f, k},
119   If[Head[r] != Root || ! MemberQ[{2, 3}, Length[r]], Return[False]];
120   If[Length[r] === 3 && ! MemberQ[{0, 1}, r[[3]]], Return[False]];
121   f = r[[1]]; k = r[[2]];
122   If[Head[f] != Function || ! IntegerQ[k] || k < 1 || ! TrueQ[NumericQ[r]], Return[False]];
123   p = bounded[f[$x], "Certificate"];

```

```

124 p != $Failed && rationalPolynomialQ[p, $x] && k <= Exponent[p, $x];
125 closedCanonicalQ[e_] := Which[
126   gaussianQ[e], True,
127   Head[e] === Root, ordinaryRootQ[e],
128   Head[e] === AlgebraicNumber && Length[e] === 2,
129   ListQ[e[[2]]] && AllTrue[e[[2]], rationalQ] && closedCanonicalQ[e[[1]]],
130   AtomQ[e], False,
131   MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, closedCanonicalQ],
132   Head[e] === Power && Length[e] === 2, rationalQ[e[[2]]] && closedCanonicalQ[e[[1]]],
133   True, False];
134 opaqueExactQ[e_] := Module[{r},
135   If[FreeQ[e, _Real | Indeterminate | _DirectedInfinity], Return[False]];
136   If[closedCanonicalQ[e], Return[True]];
137   (* Also allows triangular/algebraic-coefficient Root representations when
138      RootReduce actually converts them to the checked rational grammar. *)
139   r = bounded[RootReduce[e], "Certificate"];
140   r != $Failed && r != e && closedCanonicalQ[r];
141 algebraicFormQ[e_] := Which[
142   gaussianQ[e], True, opaqueQ[e], opaqueExactQ[e], AtomQ[e], False,
143   MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, algebraicFormQ],
144   Head[e] === Power && Length[e] === 2, rationalQ[e[[2]]] && algebraicFormQ[e[[1]]],
145   True, False];
146 exactQ[e_] := Module[{key = HoldComplete[e], result},
147   If[KeyExistsQ[$typeMemo, key], Return[True]];
148   result = FreeQ[e, _Real | Indeterminate | _DirectedInfinity] && TrueQ[algebraicFormQ[e]];
149   (* Do not cache an unsupported/timeout result as a mathematical negative. *)
150   If[result, AssociateTo[$typeMemo, key -> True]; result];
151 ExactAlgebraicQ[e_] := standalone[exactQ[e], False];
152 RadicalDepth[e_] := Which[AtomQ[e] || opaqueQ[e], 0,
153   MatchQ[e, Power[_ , _Rational]], 1 + RadicalDepth[First[e]],
154   True, Max[Prepend[RadicalDepth /@ (List @@ e), 0]]];
155 radicalNodes[e_] := Which[AtomQ[e] || opaqueQ[e], 0,
156   MatchQ[e, Power[_ , _Rational]], 1 + radicalNodes[First[e]],
157   True, Total[radicalNodes /@ (List @@ e)]];
158 bitSize[e_] := Total[Cases[e, n_Integer -> 1 + IntegerLength[Abs[n], 2], {0, Infinity}, Heads -> False] +
159   Total[Cases[e, r_Rational -> 2 + IntegerLength[Abs[Numerator[r]], 2] + IntegerLength[Denominator[r], 2],
160     {0, Infinity}, Heads -> False];
161 RadicalCost[e_] := {Count[e, _Root | _AlgebraicNumber, {0, Infinity}], RadicalDepth[e],
162   radicalNodes[e], LeafCount[e], bitSize[e]};
163 cheaperQ[a_, b_] := Order[RadicalCost[a], RadicalCost[b]] === 1;
164 smallQ[e_] := If[LeafCount[e] > $cfg["MaxLeafCount"], limitHit["MaxLeafCount"]; False, True];
165 progressQ[a_, b_] := a != b && (cheaperQ[a, b] ||
166   ($cfg["ListProgress"] === "Independent" && ListQ[a] && ListQ[b] && Length[a] === Length[b] &&
167     And @@ MapThread[{#1 === #2 || progressQ[#1, #2]} &, {a, b}]]);
168
169 (* Exact equality and heuristic rejection have different interfaces. *)
170 recordStatus[s_] := (bump["Certificates" <> s]; s);
171 certify[a_, b_] := Module[{key = HoldComplete[a, b], d, r, z, result = "Unknown"},
172   bump["Certificates"];
173   If[KeyExistsQ[$seqMemo, key], bump["CertificateCacheHits"]; Return[recordStatus[$seqMemo[key]]];
174   If[! exactQ[a] || ! exactQ[b], Return[recordStatus["Unknown"]];
175   If[a === b, AssociateTo[$seqMemo, key -> "Equal"]; Return[recordStatus["Equal"]];
176   d = bounded[a - b, "Certificate"];
177   If[d === $Failed, Return[recordStatus["Unknown"]];
178   r = bounded[RootReduce[d], "Certificate"];
179   If[r === 0, result = "Equal",
180     If[r != $Failed && gaussianQ[r] && r != 0, result = "Different",
181       (* Handles nonzero forms such as 2 Sqrt[2], not just atomic Root objects. *)
182       z = bounded[PossibleZeroQ[d, Method -> "ExactAlgebraics"], "Certificate"];
183       result = Which[z === True, "Equal", z === False, "Different", True, "Unknown"]];
184   If[result != "Unknown", AssociateTo[$seqMemo, key -> result];
185   recordStatus[result]];
186 numericallyDifferentQ[d_] := Module[{v},
187   If[! TrueQ[$cfg["NumericPrefilter"]], Return[False]];
188   v = bounded[N[d, 40]];
189   TrueQ[NumberQ[v] && Accuracy[v] >= 25 && Abs[v] > 10^-20];
190 EqualityStatus[a_, b_] := standalone[certify[a, b], "Unknown"];
191 CertifiedEqualQ[a_, b_] := EqualityStatus[a, b] === "Equal";
192 accept[c_, target_, incumbent_, method_String] := Module[{}],
193 If[c === $Failed || c === target || ! smallQ[c] || ! RadicalExpressionQ[c] || ! cheaperQ[c, incumbent],

```

```

194   Return[incumbent]];
195   If[numericallyDifferentQ[c - target], bump["NumericRejections"]; Return[incumbent]];
196   If[certify[c, target] != "Equal", Return[incumbent]];
197   bump["CandidatesAccepted"];
198   If[Length[$records] < $cfg["MaxTraceEntries"],
199     AppendTo[$records, <|"Before" -> target, "After" -> c, "Method" -> method,
200       "Scope" -> "AcceptedSearchCandidate", "EqualityEvidence" -> "ExactKernel"|>], $recordsTruncated = True];
201   trace["AcceptedCandidate", <|"Method" -> method, "Cost" -> RadicalCost[c]|>];
202   log[method, ":", target, " -> ", c]; c];
203   acceptWithPolish[c_, target_, incumbent_, method_String] := Module[{best = incumbent, v},
204     If[c == $Failed || ! smallQ[c] || ! exactQ[c], Return[best]];
205     If[numericallyDifferentQ[c - target], bump["NumericRejections"]; Return[best]];
206     best = accept[c, target, best, method];
207     v = bounded[Simplify[c, Assumptions -> True, TimeConstraint -> 5]];
208     best = accept[v, target, best, method <> "/Simplify"];
209     best = accept[bounded[Expand[c]], target, best, method <> "/Expand"];
210     best = accept[bounded[Together[c]], target, best, method <> "/Together"];
211     best = accept[rationalizeRaw[c], target, best, method <> "/Rationalize"]; best];
212
213   (* Polynomial inverse of a nonzero denominator; no sentinel polynomial. *)
214   validPolynomialQ[p_, x_, minimum_Integer: 1] := Module[{degree},
215     If[! FreeQ[p, $Failed | $Aborted | operationFailed | operationTimedOut] || ! TrueQ[PolynomialQ[p, x]], Return[
216       False]];
217     degree = Exponent[p, x]; IntegerQ[degree] && minimum <= degree <= $cfg["MaxDegree"];
218     rationalizeRaw[e_] := Module[{t, num, den, p, c, inv, result},
219       t = bounded[Together[e]]; If[t == $Failed, Return[$Failed]];
220       num = Numerator[t]; den = Denominator[t]; If[rationalQ[den], Return[t]];
221       If[! smallQ[den] || ! exactQ[den], Return[$Failed]];
222       p = bounded[MinimalPolynomial[den, $x]];
223       If[! validPolynomialQ[p, $x], Return[$Failed]];
224       c = CoefficientList[p, $x]; If[! AllTrue[c, rationalQ] || First[c] == 0, Return[$Failed]];
225       inv = bounded[-Fold[#1 den + #2 &, Last[c], Reverse[Rest[Most[c]]]]/First[c]];
226       If[inv == $Failed || certify[den inv, 1] != "Equal", Return[$Failed]];
227       result = bounded[Expand[num inv]];
228       If[result == $Failed || ! smallQ[result], $Failed, result]];
229   RationalizeDenominator[e_] := standalone[Replace[rationalizeRaw[e], $Failed -> e], e];
230   Factorc[e_] := standalone[If[exactQ[e], accept[bounded[Factor[e]], e, e, "Factor"], e], e];
231
232   (* Direct, indirect, Gaussian, and trace-norm fast paths. *)
233   quadraticParts[rho_] := Module[{e, terms, rat, irr, a, t, c, b},
234     e = bounded[Expand[rho]]; If[e == $Failed, Return[$Failed]];
235     terms = If[Head[e] == Plus, List @@ e, {e}]; rat = Select[terms, rationalQ];
236     irr = Select[terms, ! rationalQ[#] &]; If[Length[irr] != 1, Return[$Failed]];
237     a = Total[rat]; t = First[irr];
238     c = Replace[t, {Sqrt[cc_?rationalQ] -> cc, Times[k_?rationalQ, Sqrt[cc_?rationalQ]] -> k^2 cc, _ -> $Failed}];
239     If[c == $Failed || ! TrueQ[c > 0] || rationalQ[Sqrt[c]], Return[$Failed]];
240     b = Replace[t, {Sqrt[_] -> 1, Times[k_?rationalQ, Sqrt[_] -> Sign[k], _ -> 1}]; {a, b, c}];
241   quadraticSquareRoots[{a_, b_, c_}] := Module[{d = a^2 - b^2 c, s, u, v, e, out = {}},
242     If[d >= 0, s = Sqrt[d]; If[rationalQ[s] && a > 0,
243       u = (a + s)/2; v = (a - s)/2; If[u >= 0 && v >= 0, AppendTo[out, Sqrt[u] + Sign[b] Sqrt[v]]];
244       If[d < 0 && b > 0, e = Sqrt[b^2 - a^2/c]; If[rationalQ[e] && 0 <= e <= b,
245         AppendTo[out, c^(1/4) (Sqrt[(b + e)/2] + Sign[a] Sqrt[(b - e)/2])]; out];
246   rationalRoots[p_] := Module[{fl}, fl = bounded[FactorList[p]];
247     If[fl == $Failed || ! ListQ[fl], Return[{}]];
248     DeleteDuplicates[Select[Cases[fl, {f_, _Integer} /; PolynomialQ[f, $x] && Exponent[f, $x] == 1 ->
249       -Coefficient[f, $x, 0]/Coefficient[f, $x, 1]], rationalQ]];
250   cubicInQuadratic[{a_, b_, c_}] := Module[{norm, n, roots, out = {}, den, beta},
251     norm = a^2 - b^2 c; n = Sign[norm] Abs[norm]^(1/3); If[! rationalQ[n], Return[{}]];
252     roots = rationalRoots[$x^3 - 3 n $x - 2 a];
253     Do[den = t^2 - n; If[den != 0, beta = t/2 + b Sqrt[c]/den;
254       If[certify[beta^3, a + b Sqrt[c]] == "Equal", AppendTo[out, beta]], {t, roots}]; out];
255   (* Odd-index trace-norm reconstruction by Dickson polynomials. The two
256     polynomials are D_q(T,n)=u^q+v^q and S_q(T,n)=(u^q-v^q)/(u-v),
257     for u+v=T and uv=n. No divisors of an integer norm are enumerated. *)
258   quadraticOddRoots[{a_, b_, c_}, q_Integer] := Module[{norm, n, polys, roots, den, beta, out = {}},
259     If[q < 3 || ! OddQ[q] || q > $cfg["MaxDegree"], Return[{}]];
260     norm = a^2 - b^2 c; n = Sign[norm] Abs[norm]^(1/q);
261     If[! rationalQ[n], Return[{}]];
262     polys = bounded[Module[{d0 = 2, d1 = $x, s0 = 0, s1 = 1, d2, s2},
263       Do[d2 = Expand[$x d1 - n d0]; s2 = Expand[$x s1 - n s0];

```



```

263 {d0, d1} = {d1, d2}; {s0, s1} = {s1, s2}, {j, 2, q}}; {d1, s1}}];
264 If[! ListQ[polys], Return[{}]];
265 roots = rationalRoots[polys[[1]] - 2 a];
266 Do[den = polys[[2]] /. $x -> t;
267   If[den != 0, beta = t/2 + b Sqrt[c]/den;
268     If[certify[beta^q, a + b Sqrt[c]] == "Equal", AppendTo[out, beta]], {t, roots}]; out];
269 gaussianSquareRoots[z_] := Module[{a = Re[z], b = Im[z], n},
270   n = Sqrt[a^2 + b^2]; If[rationalQ[n], {Sqrt[(n + a)/2] + I Sign[b] Sqrt[(n - a)/2]}, {}]];
271 honsbeekSquareRoots[rho_] := Module[{terms, a, b, roots, den, num, out = {}},
272   If[Head[rho] != Plus || Length[rho] != 2, Return[{}]];
273   terms = List @@ rho;
274   If[! AllTrue[terms, exactQ], Return[{}]];
275   {a, b} = (bounded[RootReduce[#^3], "Certificate"] &) /@ terms;
276   If[! AllTrue[{a, b}, rationalQ] || a == 0 || b == 0, Return[{}]];
277   roots = rationalRoots[$x^4 + 4 $x^3 + 8 (b/a) $x - 4 b/a];
278   Do[den = b - s^3 a;
279     If[den != 0, num = bounded[-s^2 terms[[1]]^2/2 + s terms[[1]] terms[[2]] + terms[[2]]^2];
280     If[num != $Failed, out = Join[out, {num/Sqrt[den], -num/Sqrt[den]}]], {s, roots}];
281   (* Rational summands and negative den are allowed; the final gate chooses the branch. *)
282   out];
283
284 (* Square-class coefficient arithmetic. Every represented radicand is positive
285    and squarefree. sqrt(d_i) sqrt(d_j) = gcd(d_i, d_j) sqrt(d_i d_j / gcd^2).
286    The optional coset search is complete only in the explicitly enumerated
287    multiquadratic ambient field, and only when every rational system is solved. *)
288 integerSurdTerm[t_] := Replace[t, {
289   r_?rationalQ -> {r, 1}, Power[n_Integer, Rational[1, 2]] /; n > 0 -> {1, n},
290   Times[c_?rationalQ, Power[n_Integer, Rational[1, 2]]] /; n > 0 -> {c, n}, _ -> $Failed}];
291 squarefreeData[n_Integer] := Module[{f = bounded[FactorInteger[n]]},
292   If[! ListQ[f], Return[$Failed]];
293   {Times @@ (#[[1]]^Quotient[#[[2]], 2] & /@ f),
294    Times @@ (#[[1]]^Mod[#[[2]], 2] & /@ f), First /@ f}];
295 surdSystem[U_List, coeff_Association] := Module[{vars, polys = <|>, d, g, h, basis, eqs, sol, cand},
296   If[Length[U] > $cfg["MaxBasisSize"] || expiredQ[], Return[{}]];
297   vars = Table[Unique["c$"], {Length[U]}];
298   Do[Do[g = GCD[U[[i]], U[[j]]]; d = U[[i]] U[[j]]/g^2;
299     AssociateTo[polys, d -> (Lookup[polys, d, 0] + If[i == j, 1, 2] g vars[[i]] vars[[j]])],
300     {j, i, Length[U]}], {i, Length[U]}];
301   basis = Union[Keys[polys], Keys[coeff]];
302   eqs = (Lookup[polys, #, 0] == Lookup[coeff, #, 0] &) /@ basis;
303   bump["SquareClassSystems"]; sol = bounded[Solve[eqs, vars, Rationals]];
304   If[! ListQ[sol] || sol == {}, Return[{}]];
305   cand = vars . Sqrt[U]; DeleteDuplicates[Select[cand /. sol, RadicalExpressionQ]];
306   maskOf[n_Integer, primes_List] := Sum[If[Mod[n, primes[[i]]] == 0, 2^(i - 1), 0], {i, Length[primes]}];
307   fromMask[m_Integer, primes_List] := Times @@ Table[If[BitAnd[m, 2^(i - 1)] != 0, primes[[i]], 1], {i, Length[
308     primes]}];
309   cosetBases[radicands_List, primes_List] := Module[{h = {0}, masks, used = <|>, c, out = {}, rank = Length[primes]},
310     If[rank > $cfg["MaxPrimeRank"], limitHit["MaxPrimeRank"]; Return[{}]];
311     masks = maskOf[#, primes] & /@ radicands;
312     Do[h = Union[h, (BitXor[#, m] &) /@ h];
313       If[Length[h] > $cfg["MaxBasisSize"], limitHit["MaxBasisSize"]; Return[{}], {m, masks}];
314     (* No full 2^rank array is materialized. Enumeration is clipped before admission. *)
315     Do[If[expiredQ[], Break[]]; If[KeyExistsQ[used, j], Continue[]];
316       If[Length[out] >= $cfg["MaxCosets"], limitHit["MaxCosets"]; Break[]];
317       c = Sort[(BitXor[j, #] &) /@ h];
318       Do[AssociateTo[used, k -> True], {k, c}];
319       AppendTo[out, Sort[fromMask[#, primes] & /@ c]], {j, 0, 2^rank - 1}]; out];
320   multiSurdSquareRoots[rho_] := Module[{e, terms, parsed, coeff = <|>, primes = {}, data, surds, sets, out},
321     e = bounded[Expand[rho]]; If[e == $Failed, Return[{}]];
322     terms = If[Head[e] == Plus, List @@ e, {e}]; parsed = integerSurdTerm /@ terms;
323     If[MemberQ[parsed, $Failed], Return[{}]];
324     Do[data = squarefreeData[t[[2]]]; If[data == $Failed, Return[{}]];
325       AssociateTo[coeff, data[[2]] -> (Lookup[coeff, data[[2]], 0] + t[[1]] data[[1]])];
326       primes = Union[primes, data[[3]], {t, parsed}];
327       surds = DeleteCases[Keys[coeff], 1]; If[Length[surds] < 2 || Length[surds] > 15, Return[{}]];
328       sets = DeleteDuplicates[{Union[{1}, primes], Union[{1}, primes, surds]}];
329       Do[out = surdSystem[u, coeff]; If[out != {}, Return[out]], {u, sets}];
330       If[! TrueQ[$cfg["SquareClassSearch"]], Return[{}]];
331       sets = cosetBases[surds, primes];

```

```

331 Do[out = surdSystem[u, coeff]; If[out != {}, Return[out]], {u, sets}]; {}];
332
333 unity[k_Integer, q_Integer] := (-1)^(Mod[2 k, 2 q]/q);
334 rootCandidates[rho_, q_Integer] := Module[{parts, out = {}, r = rho, neg},
335   If[q === 2 && Head[rho] === Complex, Return[gaussianSquareRoots[rho]]];
336   neg = TrueQ[bounded[rho < 0, "Certificate"]]; If[neg, r = -rho];
337   Which[q === 2, parts = quadraticParts[r]; If[parts != $Failed, out = quadraticSquareRoots[parts]];
338     out = Join[out, honsbeekSquareRoots[r], multiSurdSquareRoots[r]]; If[neg, out = I out],
339     q === 3, parts = quadraticParts[r]; If[parts != $Failed, out = cubicInQuadratic[parts]];
340     If[neg, out = (-1)^(1/3) out],
341     OddQ[q] && q >= 5 && TrueQ[$cfg["HigherOddQuadratic"]],
342     parts = quadraticParts[r]; If[parts != $Failed, out = quadraticOddRoots[parts, q]];
343     If[neg, out = (-1)^(1/q) out]; out];
344 radicalExtension[e_] := Module[{ext = DeleteDuplicates[Cases[e, Power[_ , _Rational], {0, Infinity}]]},
345   If[! FreeQ[e, Complex], ext = Prepend[ext, I]; If[ext === {}, Automatic, ext]];
346 radicalForms[z_] := Module[{out = {z}, r},
347   If[FreeQ[z, _Root | _AlgebraicNumber], Return[out]];
348   r = bounded[ToRadicals[z]]; If[r != $Failed, AppendTo[out, r]];
349   r = bounded[RootReduce[z], "Certificate"];
350   If[r != $Failed, r = bounded[ToRadicals[r]]; If[r != $Failed, AppendTo[out, r]];
351   DeleteDuplicates[Select[out, exactQ]];
352 linearRoots[rho_, k_Integer] := Module[{fl, roots},
353   If[k > $cfg["MaxRootIndex"] || k > $cfg["MaxDegree"], limitHit["KnownPolynomialDegree"]; Return[{}]];
354   fl = bounded[FactorList[$x^k - rho, Extension -> radicalExtension[rho]]];
355   If[! ListQ[fl], fl = bounded[FactorList[$x^k - rho, Extension -> Automatic]];
356   If[! ListQ[fl], Return[{}]];
357   roots = Cases[fl, {f_, _Integer} /; PolynomialQ[f, $x] && Exponent[f, $x] === 1 :>
358     bounded[Together[-Coefficient[f, $x, 0]/Coefficient[f, $x, 1]]];
359   DeleteDuplicates[Select[Flatten[radicalForms /@ DeleteCases[roots, $Failed]], exactQ]];
360 recurse[sub_] := If[$recursion >= $cfg["MaxRecursion"] || expiredQ[], sub,
361   Block[{ $recursion = $recursion + 1, $cfg = Append[$cfg, "MaxTrials" -> Quotient[$cfg["MaxTrials"], 4]]},
362     improveNumber[sub]]];
363 negativeRadicandCandidate[target_, rho_, p_Integer, q_Integer, incumbent_] := Module[{raw, sub, best = incumbent},
364   If[! TrueQ[bounded[rho < 0, "Certificate"]], Return[best]];
365   raw = (-rho)^(p/q);
366   best = acceptWithPolish[bounded[Expand[(-1)^(p/q) raw], target, best, "NegativePhase"];
367   sub = recurse[raw];
368   acceptWithPolish[bounded[Expand[(-1)^(p/q) sub], target, best, "NegativeRadicand"]];
369 kummerCandidates[target_, rho_, p_Integer, q_Integer, incumbent_] := Module[
370   {best = incumbent, gammas, rest, den, raw, sub},
371   Do[If[expiredQ[], Break[]]; gammas = linearRoots[rho, k]; rest = q/k;
372     Do[If[expiredQ[], Break[]];
373       If[rest === 1,
374         Do[best = acceptWithPolish[bounded[Expand[(gamma unity[l, q])^p], target, best, "LinearFactor"], {l, 0, q -
375           1}]],
376         Do[den = bounded[Expand[gamma unity[j, k]]]; If[den === $Failed, Continue[]];
377         raw = den^(1/rest);
378         (* Offer the raw index reduction even when recursion does nothing. *)
379         Do[best = acceptWithPolish[bounded[Expand[(raw unity[l, rest])^p], target, best, "IndexReduction"], {l, 0,
380           rest - 1}];
381         sub = recurse[raw];
382         If[sub != raw, Do[best = acceptWithPolish[bounded[Expand[(sub unity[l, rest])^p], target, best,
383           "IndexReduction/Recursive"], {l, 0, rest - 1}]], {j, 0, k - 1}]], {gamma, gammas}];
384         If[RadicalDepth[best] <= 1, Break[]], {k, Reverse[Rest[Divisors[q]]]]; best];
385 (* Bounded FIFO multiplier search, including cubic/quartic GCD factors. *)
386 reductionIndex[e_] := Module[{d = RadicalDepth[e], nodes, q = 1},
387   nodes = Select[Cases[e, Power[_ , _Rational], {0, Infinity}], RadicalDepth[#] === d &];
388   Do[q = LCM[q, Denominator[Last[node]]];
389     If[q > $cfg["MaxRootIndex"], limitHit["MaxRootIndex"]; Return[$Failed], {node, nodes}];
390     If[q < 2, $Failed, q]];
391 complementaryFactor[Power[b_, r_Rational]] := b^(Mod[-Numerator[r], Denominator[r]]/Denominator[r]);
392 complementaryFactor[Complex[0, _]] := -I;
393 complementaryFactor[e_] := Module[{deep = Cases[e, Power[_ , _Rational], {0, Infinity}], maxd},
394   If[deep === {}, 1, maxd = Max[RadicalDepth /@ deep];
395   Times @@ (complementaryFactor /@ Select[deep, RadicalDepth[#] === maxd &])];
396 complementaryMultiplier[t_] := Times @@ (complementaryFactor /@ If[Head[t] === Times, List @@ t, {t}]);
397 multiplierSearch[target_, initialBest_] := Module[
398   {best = initialBest, q, rho, queue = {}, seen = <||>, admitted = 0, proposed = 0, cursor = 1,
399   cap = $cfg["MultiplierCap"], proposalCap, admit, seeds, terms, m, theta, poly, degree, gcd, gd,

```



```

399 roots, mroot, candidate, disc, primes, factorization, j, digits, batch, sol, before,
400 keyTarget = HoldComplete[target], sinceImprovement = 0, bestDegree = Infinity},
401 If[cap === 0 || $cfg["MaxTrials"] === 0 || expiredQ[], Return[best]];
402 q = reductionIndex[target]; If[q === $Failed, Return[best]];
403 rho = bounded[Expand[target^q]]; If[rho === $Failed || ! exactQ[rho], Return[best]];
404 proposalCap = 4 cap;
405 admit[value_] := Module[{canonical, key},
406   If[admitted >= cap, limitHit["MultiplierCap"]; Return[False]];
407   If[proposed >= proposalCap, limitHit["MultiplierProposals"]; Return[False]];
408   proposed++; bump["MultipliersProposed"];
409   If[! smallQ[value] || ! exactQ[value], Return[False]];
410   canonical = bounded[Expand[value]];
411   If[canonical === $Failed || canonical === 0, Return[False]];
412   If[TrueQ[$cfg["CanonicalMultipliers"]],
413     canonical = bounded[RootReduce[canonical], "Certificate"];
414     If[canonical === $Failed || canonical === 0, Return[False]];
415     (* Full expressions as keys avoid formatting as a hidden normalization step.
416        Default equivalence is expanded syntax, not equality of algebraic values. *)
417     key = With[{v = canonical}, HoldComplete[v]];
418     If[KeyExistsQ[seen, key], bump["DuplicateMultipliers"]; Return[False]];
419     AssociateTo[seen, key -> True]; AppendTo[queue, value]; admitted++; bump["MultipliersAdmitted"]; True];
420 If[ListQ[$cfg["Multipliers"]], seeds = $cfg["Multipliers"],
421   terms = DeleteCases[Replace[#, {
422     Times[a___, _?rationalQ, b___] :> a*b, _?rationalQ -> 0, Complex[a_, b_] :> Sign[b] I}]] & /@
423     If[Head[rho] === Plus, List @@ rho, {rho}], 0];
424 seeds = bounded[Join[{1}, DeleteCases[complementaryMultiplier /@ terms, 1], {2, 3, 5}]];
425 If[! ListQ[seeds], seeds = {1}];
426 Do[If[admitted >= cap || proposed >= proposalCap || expiredQ[], Break[]; admit[seed], {seed, seeds}];
427 While[cursor <= Length[queue] && ! expiredQ[],
428   (* Usage is shared by repeated visits to this same target. A low-quota
429      recursive visit cannot consume the larger quota of a later outer visit. *)
430   If[Lookup[$trialUse, keyTarget, 0] >= $cfg["MaxTrials"], limitHit["MaxTrials"]; Break[]];
431   If[best != initialBest && sinceImprovement >= $cfg["Patience"], limitHit["Patience"]; Break[]];
432   m = queue[[cursor]]; cursor++; sinceImprovement++; bump["Trials"];
433   AssociateTo[$trialUse, keyTarget -> (Lookup[$trialUse, keyTarget, 0] + 1)];
434   If[certify[m, 0] != "Different", Continue[]];
435   theta = bounded[Expand[m rho]]; If[theta === $Failed, Continue[]];
436   poly = bounded[MinimalPolynomial[theta^(1/q), $x]];
437   If[! validPolynomialQ[poly, $x], If[poly != $Failed, limitHit["MaxDegree"]; Continue[]];
438   degree = Exponent[poly, $x]; bestDegree = Min[bestDegree, degree];
439   trace["Trial", <|"Multiplier" -> m, "Degree" -> degree, "Index" -> q|>];
440   roots = If[m === 1, {}, linearRoots[theta, q]];
441   gcd = bounded[PolynomialGCD[poly, $x^q - theta, Extension -> Automatic]];
442   If[validPolynomialQ[gcd, $x], gd = Exponent[gcd, $x];
443     If[gd < q, roots = Join[roots, Which[
444       gd === 1, {-Coefficient[gcd, $x, 0]/Coefficient[gcd, $x, 1]},
445       gd <= $cfg["MaxSolveDegree"], sol = bounded[Solve[gcd == 0, $x]];
446       If[ListQ[sol], Select[$x /. sol, exactQ], {}, True, {}]]];
447   roots = DeleteDuplicates[Select[Flatten[radicalForms /@ roots], exactQ]];
448   If[roots != {}, mroot = bounded[m^(1/q)];
449     If[mroot != $Failed, Do[If[expiredQ[], Break[]];
450       Do[If[expiredQ[], Break[]; candidate = bounded[Expand[z/mroot unity[k, q]]]; before = best;
451         best = acceptWithPolish[candidate, target, best, "MultiplierOrbit"];
452         If[best != before, sinceImprovement = 0];
453         If[candidate != $Failed && candidate != target && RadicalDepth[candidate] <= RadicalDepth[target] &&
454           certify[candidate, target] === "Equal", before = best;
455         best = acceptWithPolish[recurse[candidate], target, best, "MultiplierOrbit/Recursive"];
456         If[best != before, sinceImprovement = 0]], {k, 0, q - 1}], {z, roots}]]];
457   If[RadicalDepth[best] <= 1, Break[]];
458   If[! ListQ[$cfg["Multipliers"]] && degree <= bestDegree && admitted < cap && proposed < proposalCap && ! expiredQ
459     [],
460     disc = bounded[Discriminant[poly, $x]];
461     If[IntegerQ[disc] && disc != 0,
462       factorization = bounded[FactorInteger[Abs[disc]]];
463       primes = If[ListQ[factorization], Take[First /@ factorization, UpTo[8]], {}]; batch = 0;
464       Do[If[admitted >= cap || proposed >= proposalCap || batch >= 24 || expiredQ[], Break[]];
465         Do[If[admitted >= cap || proposed >= proposalCap || batch >= 24 || expiredQ[], Break[]];
466           batch++; admit[m prime^exponent], {exponent, 1, q - 1}], {prime, primes}];
467       j = 1;
468       While[primes != {} && j < q^Length[primes] && batch < 24 && admitted < cap && proposed < proposalCap && !

```

```

    expiredQ[],
468     digits = IntegerDigits[j, q, Length[primes]]; j++; batch++;
469     admit[m Times @@ MapThread[Power, {primes, digits}]]]]];
470 best];
471
472 (* Every stage preserves its incoming incumbent, including partial reductions. *)
473 powerProposals[target_, incumbent_] := Module[{best = incumbent, rho, p, q, root, raw, cand},
474   If[! MatchQ[target, Power[_], _Rational]], Return[best];
475   rho = First[target]; p = Numerator[Last[target]]; q = Denominator[Last[target]];
476   If[q > $cfg["MaxRootIndex"], limitHit["MaxRootIndex"]; Return[best];
477   cand = rootCandidates[rho, q];
478   Do[best = acceptWithPolish[bounded[Expand[c^p]], target, best, "FastPath"], {c, cand}];
479   If[RadicalDepth[best] <= 1, bump["FastPathAccepted"]; Return[best];
480   best = negativeRadicalCandidate[target, rho, p, q, best];
481   If[RadicalDepth[best] <= 1, Return[best];
482   best = kummerCandidates[target, rho, p, q, best];
483   If[RadicalDepth[best] >= 2 && EvenQ[q] && q > 2,
484     root = recurse[rho^(1/2)]; raw = root^(2/q);
485     best = acceptWithPolish[bounded[Expand[raw^p]], target, best, "EvenIndexSplit"];
486     root = recurse[raw];
487     best = acceptWithPolish[bounded[Expand[root^p]], target, best, "EvenIndexSplit/Recursive"]; best];
488 genericProposals[target_, incumbent_] := Module[{best = incumbent, r, solver, polynomial},
489   solver = $cfg["Solver"];
490   If[solver != Automatic, r = bounded[Catch[solver[target], _, $Failed &]];
491   best = acceptWithPolish[r, target, best, "Solver"];
492   If[! FreeQ[target, _Root | _AlgebraicNumber], r = bounded[ToRadicals[target]];
493   If[r != $Failed && r != target,
494     best = acceptWithPolish[r, target, best, "ToRadicals"];
495     best = acceptWithPolish[recurse[r], target, best, "ToRadicals/Recursive"]];
496   r = bounded[RootReduce[target], "Certificate"];
497   If[r != $Failed, best = accept[r, target, best, "RootReduce"];
498   If[Head[r] === Root && ordinaryRootQ[r], polynomial = bounded[First[r][$x]];
499   If[polynomial != $Failed && Exponent[polynomial, $x] <= 4,
500     best = acceptWithPolish[bounded[ToRadicals[r]], target, best, "RootReduce/ToRadicals"]];
501   best = accept[bounded[Simplify[target, Assumptions -> True, TimeConstraint -> 5]], target, best, "Simplify"];
502   If[TrueQ[$cfg["Factor"]], best = accept[bounded[Factor[target]], target, best, "Factor"];
503   best = accept[rationalizeRaw[target], target, best, "Rationalize"]; best];
504 searchableQ[e_] := MatchQ[e, Power[_], _Rational] ||
505   (Head[e] === Times && AllTrue[List @@ e, MatchQ[#, _?gaussianQ | Power[_], _Rational] &]);
506 improveNumber[target_] := Module[{best = target, key = HoldComplete[target]},
507   If[expiredQ[] || ! smallQ[target] || ! exactQ[target], Return[target];
508   If[RadicalDepth[target] < 2 && FreeQ[target, _Root | _AlgebraicNumber], Return[target];
509   If[KeyExistsQ[$memo, key], best = $memo[key]; bump["PositiveMemoHits"];
510   If[KeyExistsQ[$inProgress, key] || (best != target && RadicalDepth[best] <= 1), Return[best];
511   AssociateTo[$inProgress, key -> True]; bump["Islands"];
512   best = powerProposals[target, best];
513   If[RadicalDepth[best] >= 2 || ! FreeQ[best, _Root | _AlgebraicNumber], best = genericProposals[target, best];
514   If[RadicalDepth[best] >= 2 && $cfg["Solver"] === Automatic && searchableQ[target] && ! expiredQ[],
515     best = multiplierSearch[target, best];
516   (* Cache certified positive progress as an incumbent, never a failed search. *)
517   If[best != target, AssociateTo[$memo, key -> best];
518   KeyDropFrom[$inProgress, key]; best];
519 combineProposals[target_, incumbent_] := Module[{best = incumbent, r},
520   r = bounded[RootReduce[incumbent], "Certificate"]; best = accept[r, target, best, "Combine/RootReduce"];
521   best = accept[bounded[Simplify[incumbent, Assumptions -> True, TimeConstraint -> 5]], target, best, "Combine/
    Simplify"];
522   best = accept[bounded[Together[incumbent]], target, best, "Combine/Together"];
523   accept[bounded[Expand[incumbent]], target, best, "Combine/Expand"];
524   island[e_] := Module[{current = e, rebuilt},
525     If[expiredQ[] || (RadicalDepth[e] < 2 && FreeQ[e, _Root | _AlgebraicNumber]), Return[e];
526     If[MatchQ[e, Power[_], _Rational],
527       If[TrueQ[$cfg["AllLevels"]], rebuilt = Power[island[First[e]], Last[e]];
528       If[rebuilt != e && cheaperQ[rebuilt, e], current = rebuilt];
529       Return[improveNumber[current]];
530     If[MemberQ[{Plus, Times}, Head[e]] || MatchQ[e, Power[_], _Integer],
531       rebuilt = If[Head[e] === Power, Power[island[First[e]], Last[e]], Map[island, e]];
532       If[rebuilt != e && cheaperQ[rebuilt, e], current = rebuilt];
533       If[RadicalDepth[current] >= 2 || ! FreeQ[current, _Root | _AlgebraicNumber], current = improveNumber[current];
534       If[current != e, current = combineProposals[e, current]; Return[current];
535     If[opaqueQ[e], Return[improveNumber[e]]; e];

```

```

536 walk[e_] := Module[{h},
537   If[expiredQ[] || ! smallQ[e], Return[e]];
538   If[exactQ[e], Return[island[e]]]; If[AtomQ[e], Return[e]]; h = Head[e];
539   Which[MemberQ[{List, Plus, Times}, h], Map[walk, e],
540     h === Power && Length[e] === 2 && rationalQ[Last[e]], Power[walk[First[e]], Last[e]], True, e];
541
542   (* Session controls cover preflight and costs as well as the search. Reports
543      after a timeout may legitimately contain Missing costs rather than spend
544      unbounded work computing them after the deadline. *)
545   run[e_, cfg_Association, coreOnly_> False] :=
546     Block[{$active = True, $cfg = cfg, $deadline = AbsoluteTime[] + cfg["TimeBudget"],
547       $stats = newStats[], $limits = <|>, $trace = {}, $records = {}, $commits = {},
548       $recordsTruncated = False, $traceTruncated = False, $commitsTruncated = False,
549       $memo = <|>, $inProgress = <|>, $typeMemo = <|>, $eqMemo = <|>, $trialUse = <|>,
550       $recursion = 0, $Assumptions = True},
551     Module[{snapshot = e, beforeSnapshot, candidate, outcome = Null, started = AbsoluteTime[], passes, status,
552       initialCost = Missing["NotComputed"], finalCost = Missing["NotComputed"], numericInput = False},
553       passes = If[TrueQ[cfg["AllLevels"]] && ! coreOnly, cfg["MaxPasses"], 1];
554       If[TrueQ[cfg["TimeBudget"]] == 0, limitHit["TimeBudget"],
555         outcome = Quiet[Check[TimeConstrained[MemoryConstrained[
556           initialCost = RadicalCost[e];
557           Which[! smallQ[e], Null,
558             ! TrueQ[cfg["ProcessExactIslands"]] && ! FreeQ[e, _Real | _Complex?InexactNumberQ], limitHit["InexactInput"],
559             coreOnly && ! exactQ[e], limitHit["NotExactAlgebraic"],
560             True, numericInput = exactQ[e];
561             Do[If[expiredQ[], Break[]]; candidate = If[coreOnly, improveNumber[snapshot], walk[snapshot]];
562             bump["PassesCompleted"];
563             If[candidate === snapshot || ! progressQ[candidate, snapshot], Break[]];
564             If[numericInput && certify[candidate, e] != "Equal", Break[]];
565             beforeSnapshot = snapshot; snapshot = candidate;
566             If[Length[$commits] < cfg["MaxTraceEntries"],
567               AppendTo[$commits, <|"Before" -> beforeSnapshot, "After" -> snapshot,
568                 "Evidence" -> If[numericInput, "ExactKernel", "ArithmeticCongruence"], "Pass" -> pass|>],
569               $commitsTruncated = True];
570             If[pass === passes && passes > 1, limitHit["MaxPasses"], {pass, passes}];
571             finalCost = RadicalCost[snapshot]; Null,
572             cfg["MemoryBudget"], memoryStopped], cfg["TimeBudget"], timeStopped], sessionFailed]]];
573       If[outcome === memoryStopped, limitHit["MemoryBudget"]];
574       If[outcome === timeStopped, limitHit["TimeBudget"]];
575       If[outcome === sessionFailed, limitHit["SessionFailure"]];
576       If[MemberQ[{memoryStopped, timeStopped, sessionFailed}, outcome],
577         $commitsTruncated = True]; (* An interrupt can precede journal append. *)
578       status = Which[TrueQ[cfg["TimeBudget"]] == 0, "Disabled", outcome === memoryStopped, "MemoryLimit",
579         outcome === timeStopped || KeyExistsQ[$limits, "TimeBudget"], "Timeout",
580         outcome === sessionFailed, "Failure", snapshot === e, "Unchanged", True, "Improved"];
581       <|"Result" -> snapshot, "Status" -> status, "ResultChanged" -> (snapshot != e),
582       "Limits" -> Keys[$limits], "Statistics" -> $stats, "InitialCost" -> initialCost, "FinalCost" -> finalCost,
583       "Certificates" -> $records, "CertificateRecordScope" -> "AcceptedSearchCandidatesNotFinalProof",
584       "CandidateRecordsTruncated" -> $recordsTruncated, "CommittedPasses" -> $commits,
585       "CommittedPassesTruncated" -> $commitsTruncated, "Trace" -> $trace, "TraceTruncated" -> $traceTruncated,
586       "ElapsedSeconds" -> AbsoluteTime[] - started, "Options" -> cfg,
587       "BudgetScope" -> "Core session; excludes argument/option evaluation and serialization",
588       "CompletenessClaim" -> False|>];
589   SetAttributes[standalone, HoldAll];
590   standalone[body_, fallback_] := If[TrueQ[$active], body,
591     Module[{cfg = resolveOptions[Strad, {}], seconds},
592       If[FailureQ[cfg], Return[fallback]]; seconds = Min[20, cfg["TimeBudget"]];
593       If[! TrueQ[seconds > 0], Return[fallback]];
594       Block[{$active = True, $cfg = cfg, $deadline = AbsoluteTime[] + seconds,
595         $stats = newStats[], $limits = <|>, $trace = {}, $records = {}, $commits = {},
596         $recordsTruncated = False, $traceTruncated = False, $commitsTruncated = False,
597         $memo = <|>, $inProgress = <|>, $typeMemo = <|>, $eqMemo = <|>, $trialUse = <|>,
598         $recursion = 0, $Assumptions = True},
599         Quiet[Check[TimeConstrained[MemoryConstrained[body, cfg["MemoryBudget"], fallback], seconds, fallback], fallback
600           ]]]];
601   invoke[e_, head_Symbol, raw_List, report_, core_, allowFlag_> True] := Module[{rules = raw, cfg, result},
602     If[allowFlag && rules != {} && booleanQ[First[rules]], rules = Prepend[Rest[rules], "AllLevels" -> First[rules]]];
603     cfg = resolveOptions[head, rules]; If[FailureQ[cfg], Return[cfg]];
604     result = run[e, cfg, core]; If[TrueQ[report], result, result["Result"]];

```

```
604 Strad[e_, args___] := invoke[e, Strad, {args}, False, False];
605 DenestRadicals[e_, args___] := invoke[e, DenestRadicals, {args}, False, False];
606 DenestCore[e_, args___] := invoke[e, DenestCore, {args}, False, True, False];
607 DenestReport[e_, args___] := invoke[e, DenestReport, {args}, True, False];
608 Strad[] := Failure["MissingInput", <||>];
609 DenestRadicals[] := Failure["MissingInput", <||>];
610 DenestCore[] := Failure["MissingInput", <||>];
611 DenestReport[] := Failure["MissingInput", <||>];
612 End[];
613 EndPackage[];
```